THE UNIVERSITY OF MELBOURNE

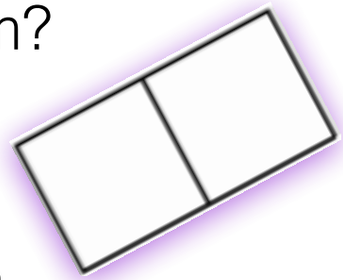
COMP90038 Algorithms and Complexity

Lecture 2: Review of Basic Concepts

(with thanks to Harald Søndergaard)
Olya Ohrimenko / Cezary Kaliszyk

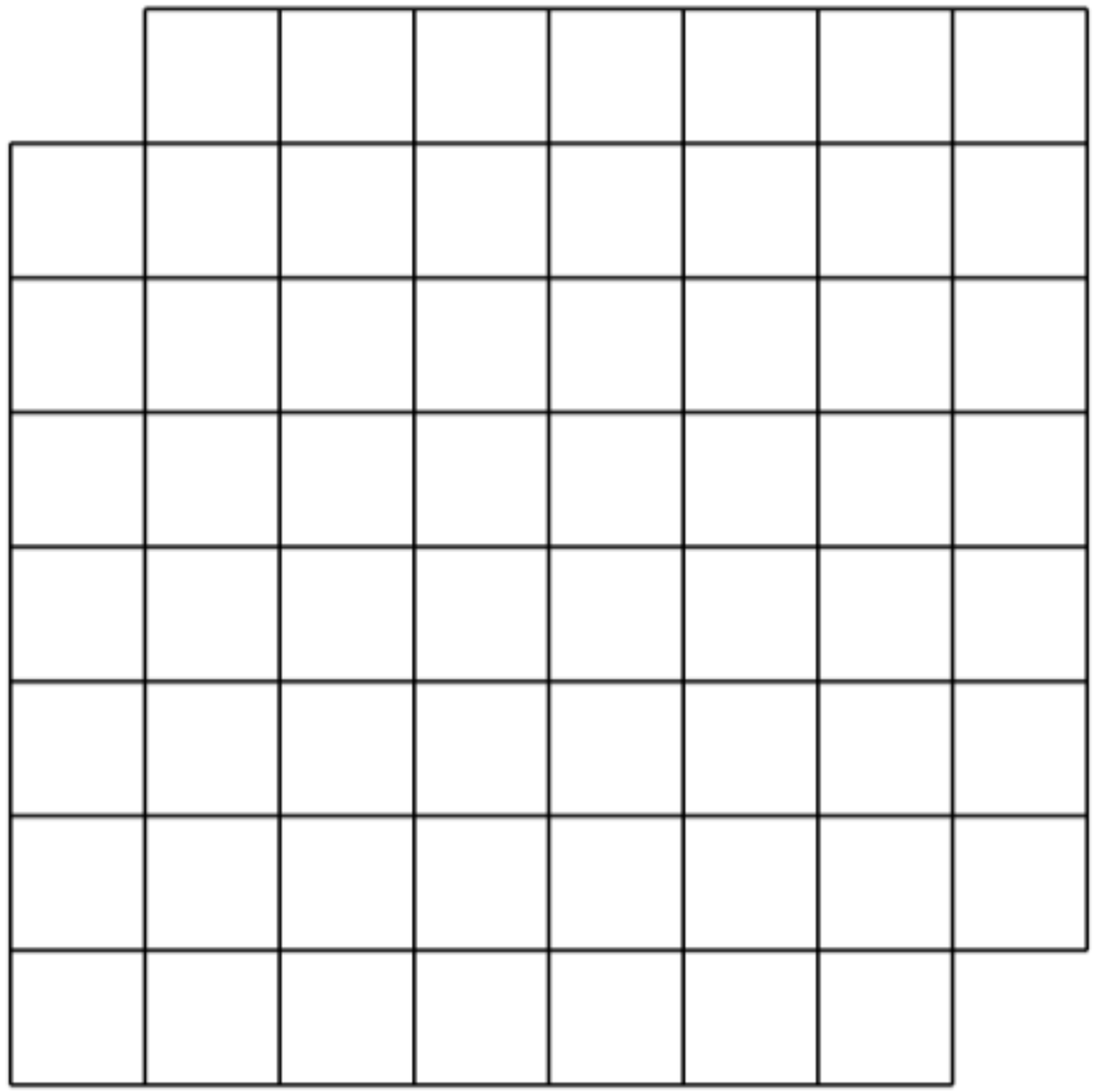
Approaching a problem

- Can we cover this board with 31 tiles of the following form?



- This is the **mutilated checkerboard problem**.

- There are only finitely many ways we can arrange the 31 tiles, so there is a brute-force (and very inefficient) way of solving the problem.



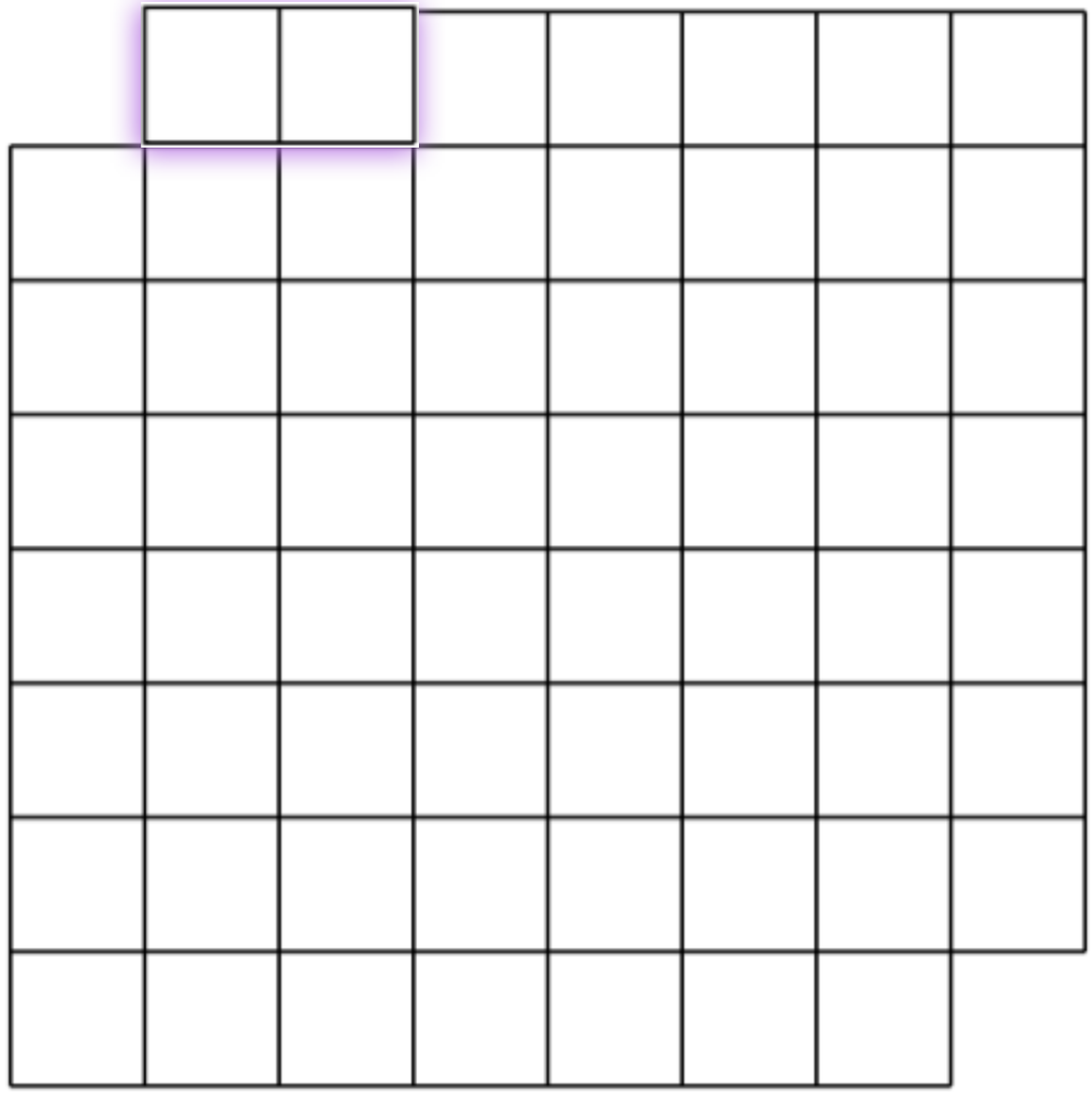
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Approaching a problem

- Can we cover this board with 31 tiles of the following form?

- This is the **mutilated checkerboard problem**.

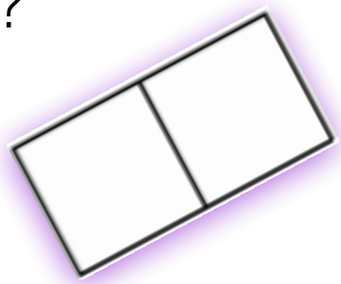
- There are only finitely many ways we can arrange the 31 tiles, so there is a brute-force (and very inefficient) way of solving the problem.



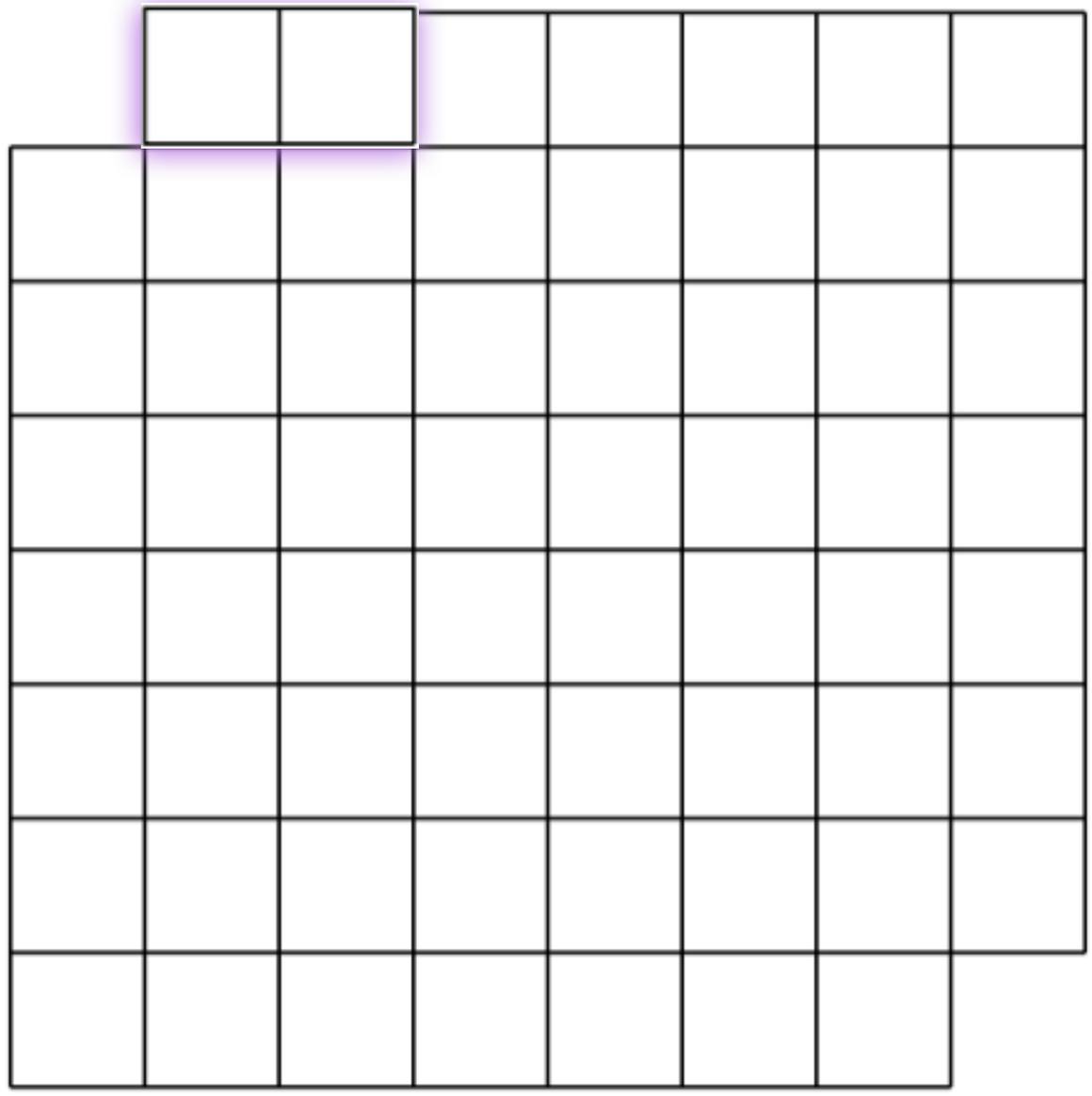
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Approaching a problem

- Can we cover this board with 31 tiles of the following form?



- This is the **mutilated checkerboard problem**.



- There are only finitely many ways we can arrange the 31 tiles, so there is a brute-force (and very inefficient) way of solving the problem.

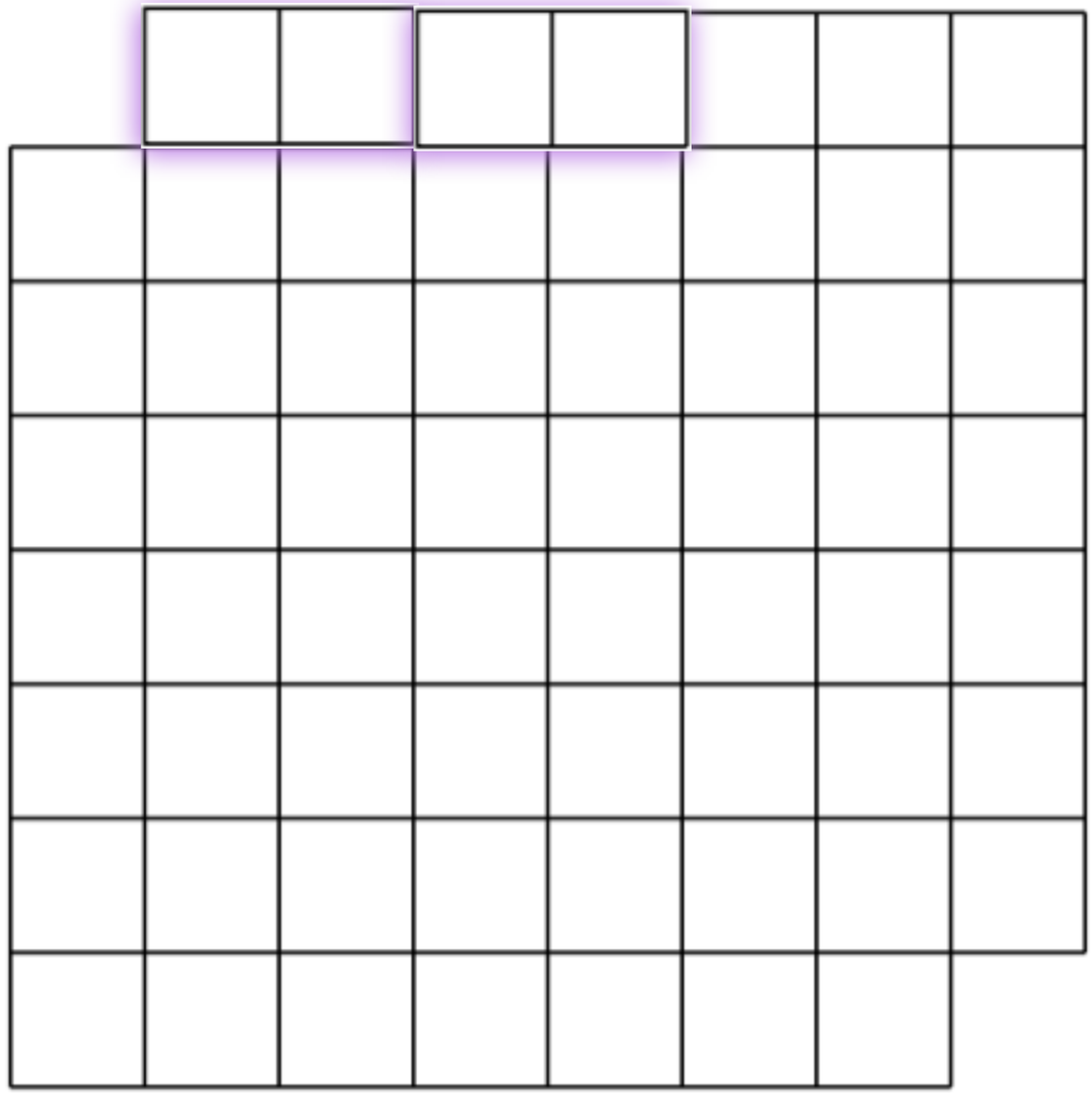
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Approaching a problem

- Can we cover this board with 31 tiles of the following form?

- This is the **mutilated checkerboard problem**.

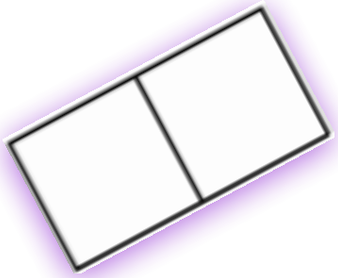
- There are only finitely many ways we can arrange the 31 tiles, so there is a brute-force (and very inefficient) way of solving the problem.



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

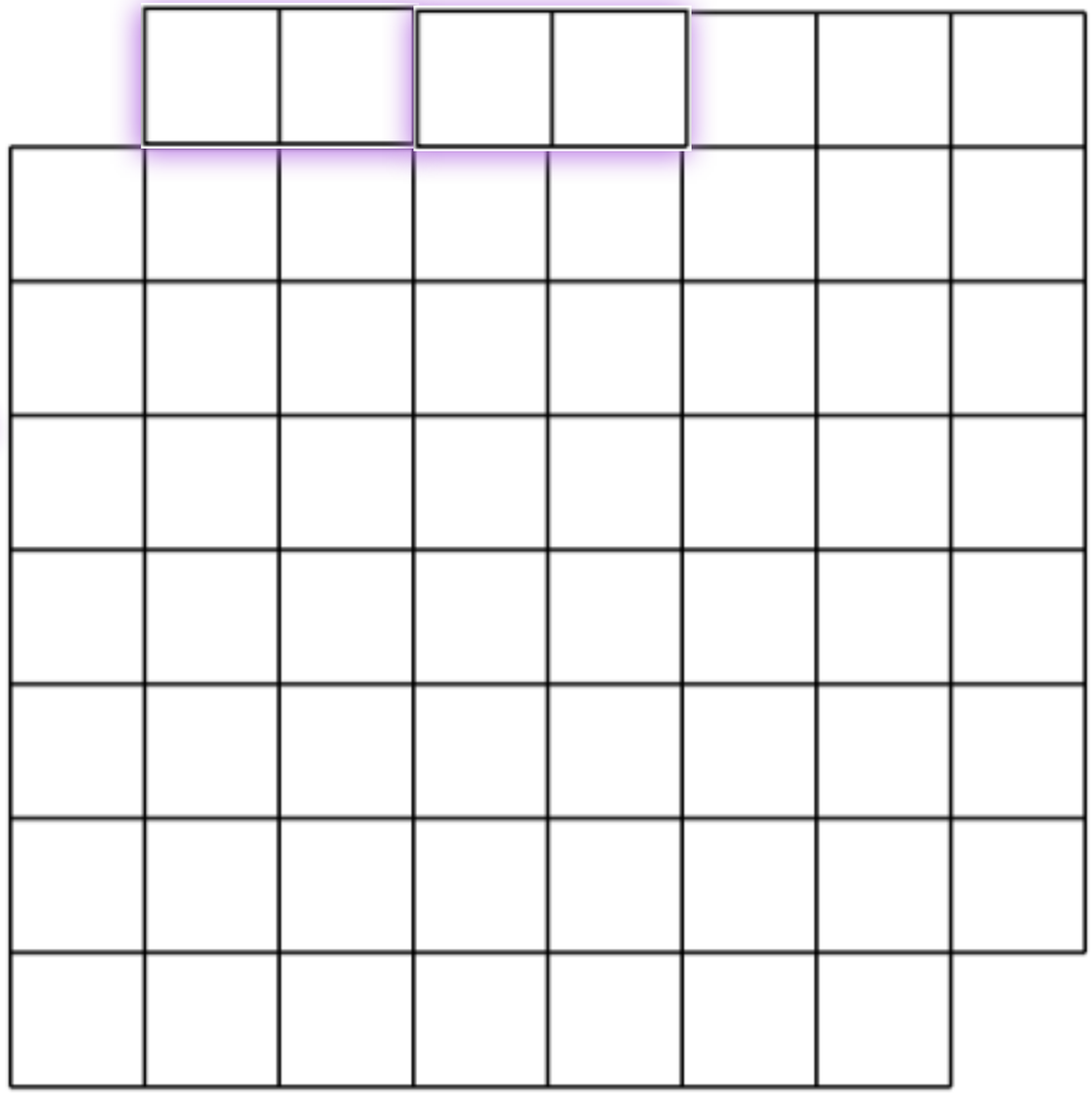
Approaching a problem

- Can we cover this board with 31 tiles of the following form?



- This is the **mutilated checkerboard problem**.

- There are only finitely many ways we can arrange the 31 tiles, so there is a brute-force (and very inefficient) way of solving the problem.



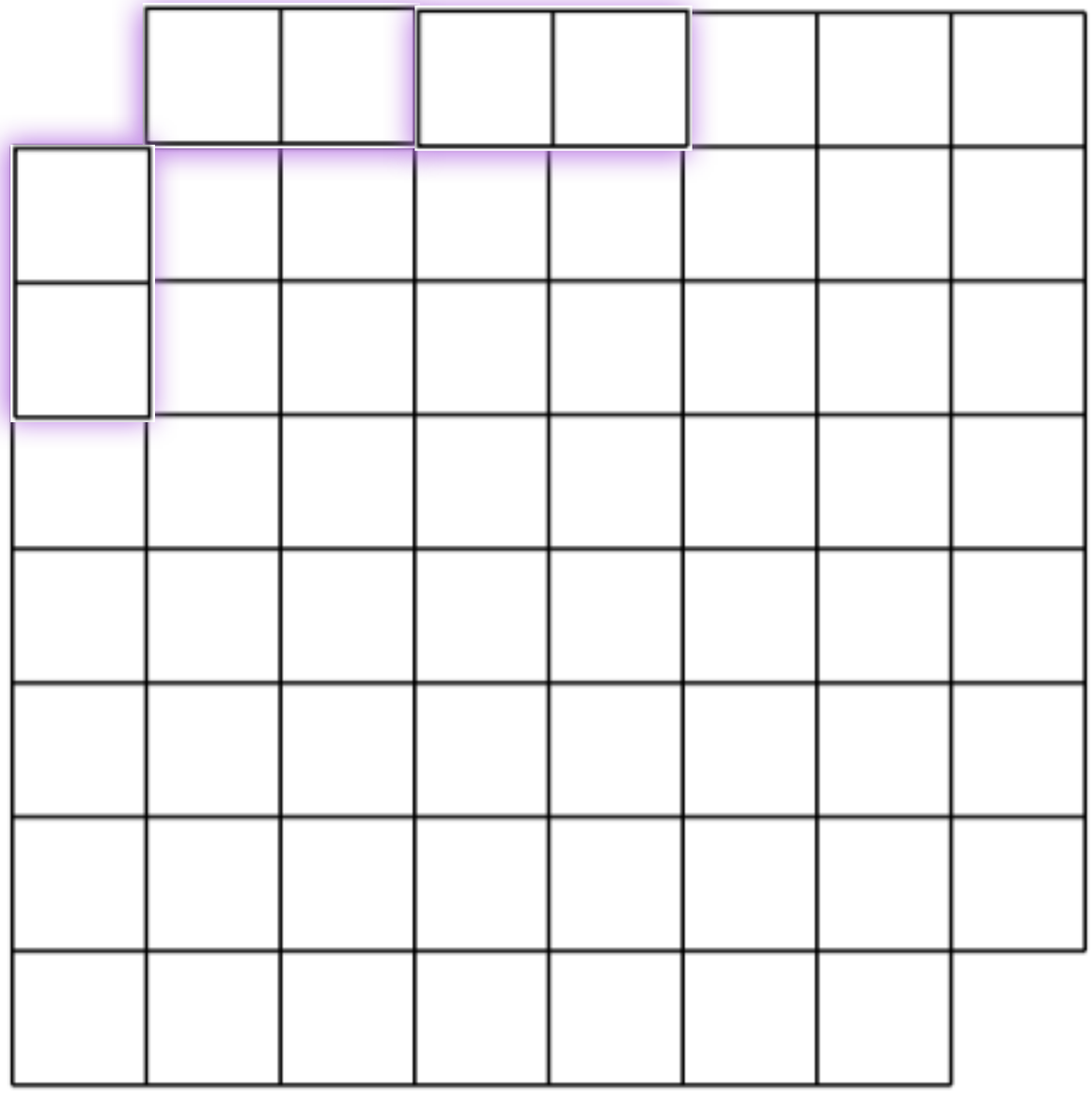
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Approaching a problem

- Can we cover this board with 31 tiles of the following form?

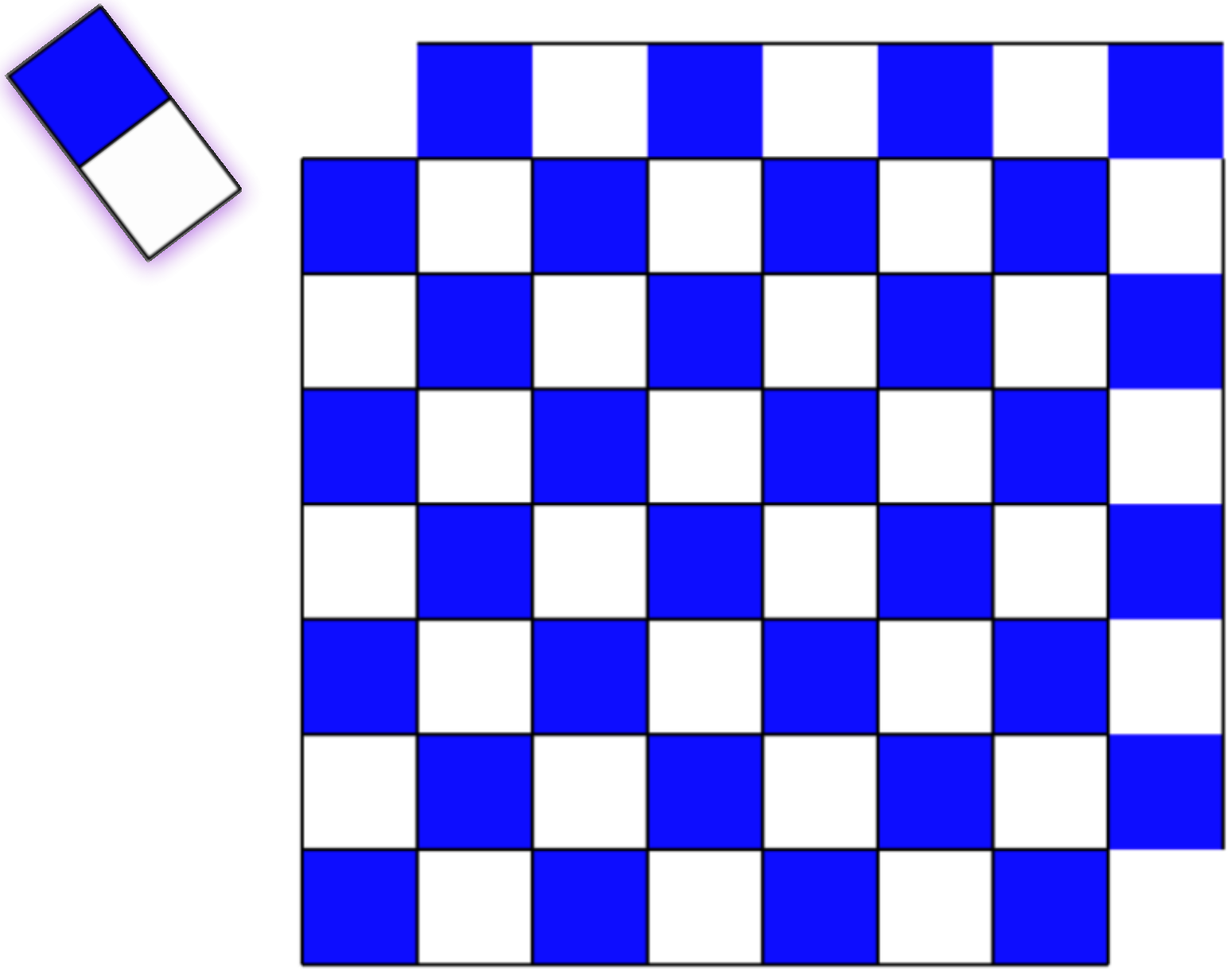
- This is the **mutilated checkerboard problem**.

- There are only finitely many ways we can arrange the 31 tiles, so there is a brute-force (and very inefficient) way of solving the problem.



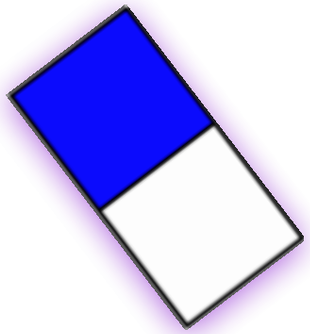
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

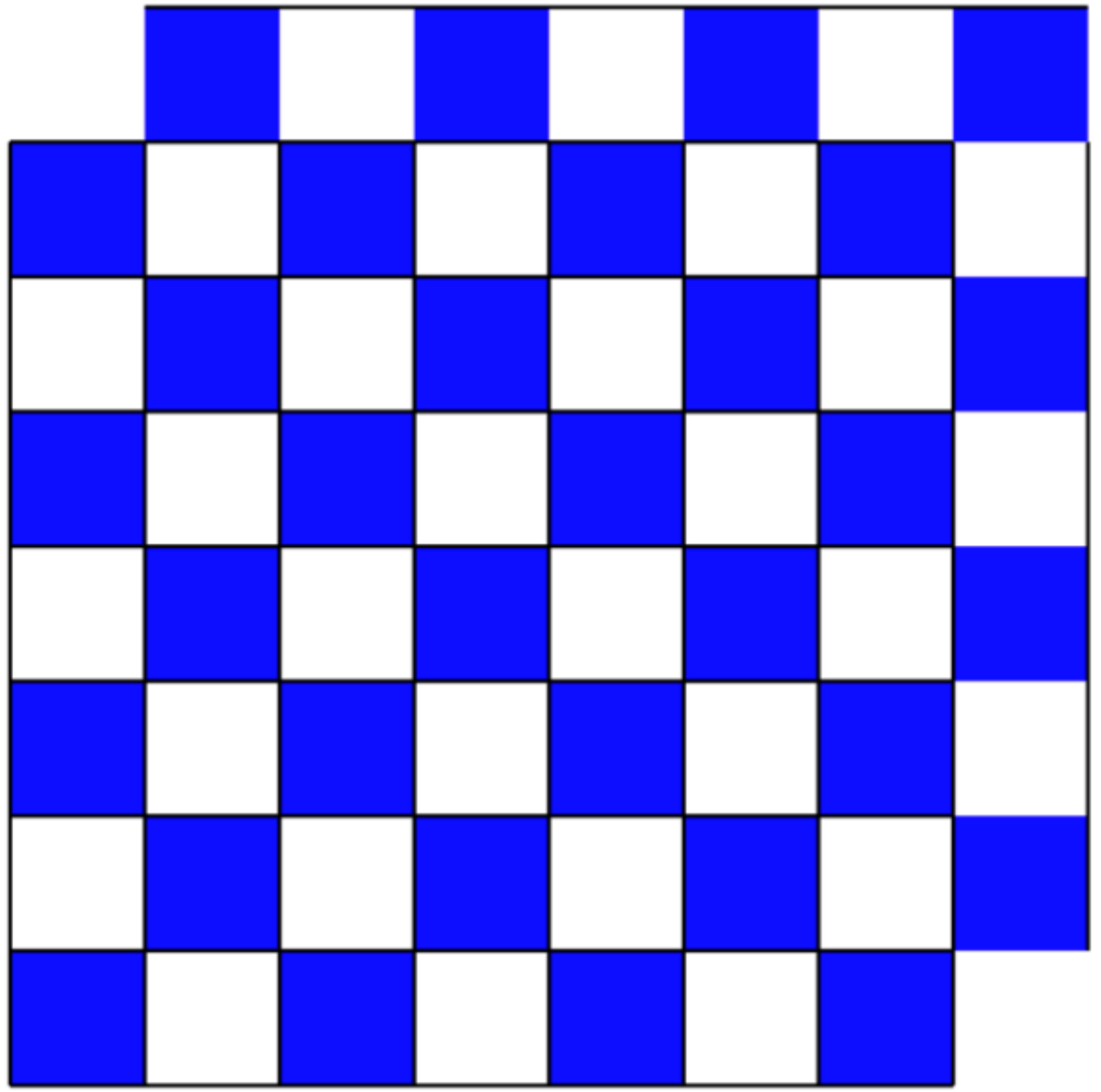
Transform and Conquer?
Use abstraction?



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Transform and Conquer? Use abstraction?

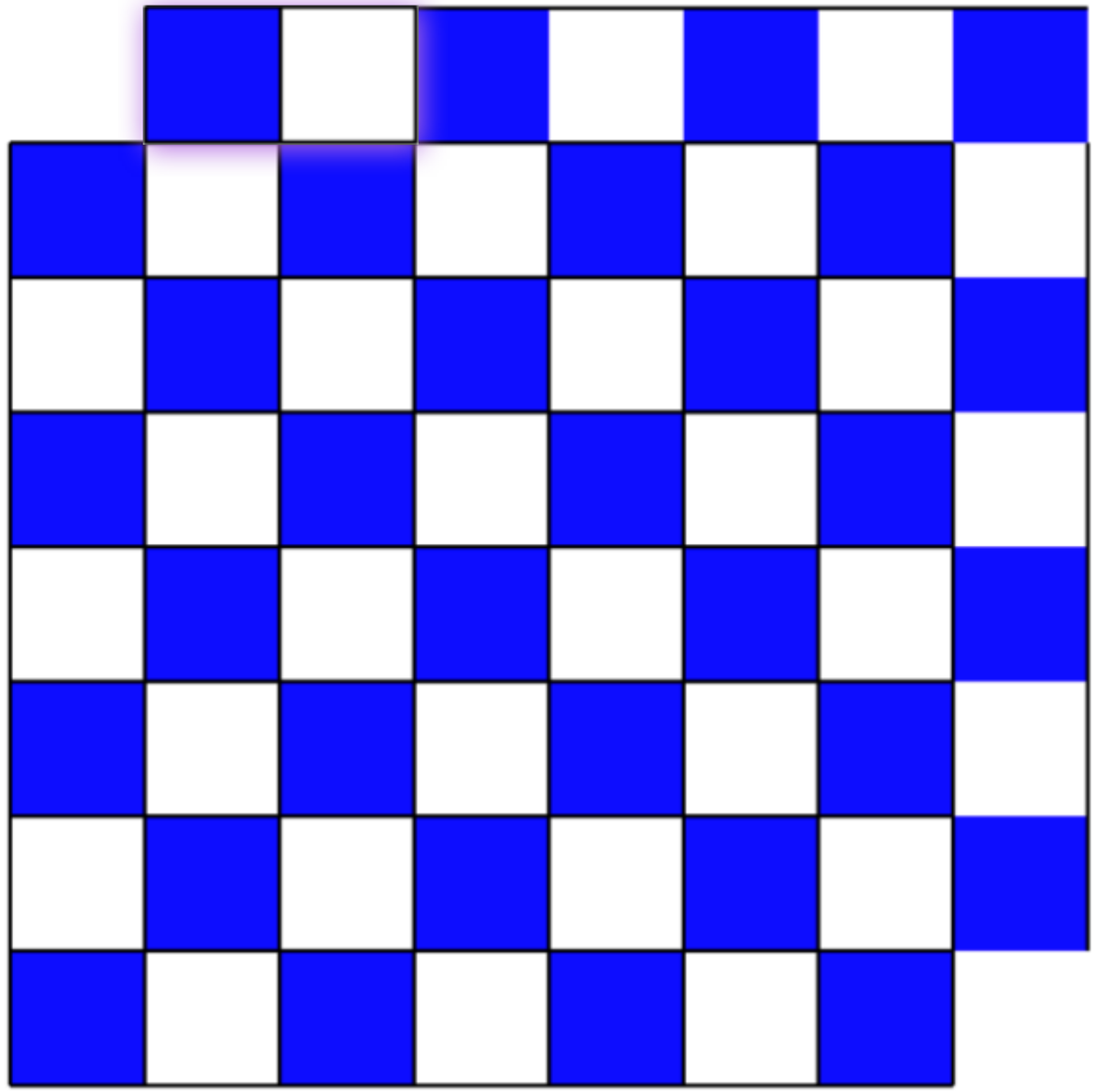




- Can we cover this board with 31 tiles of the form shown?
- Why can we quickly determine that the answer is no?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

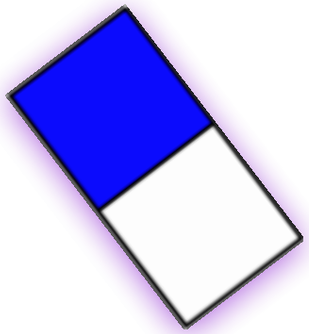
Transform and Conquer? Use abstraction?

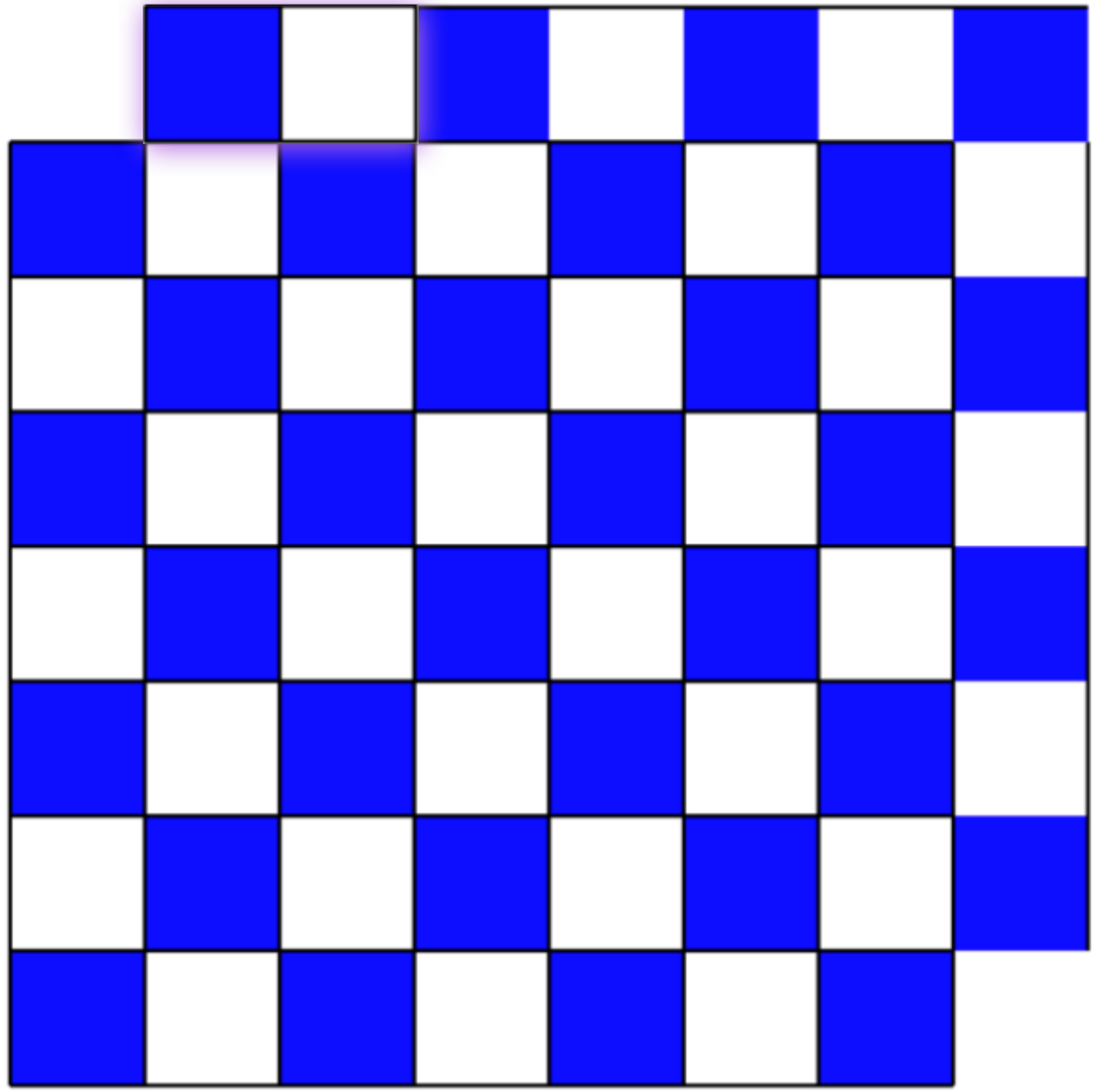


- Can we cover this board with 31 tiles of the form shown?
- Why can we quickly determine that the answer is no?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Transform and Conquer? Use abstraction?

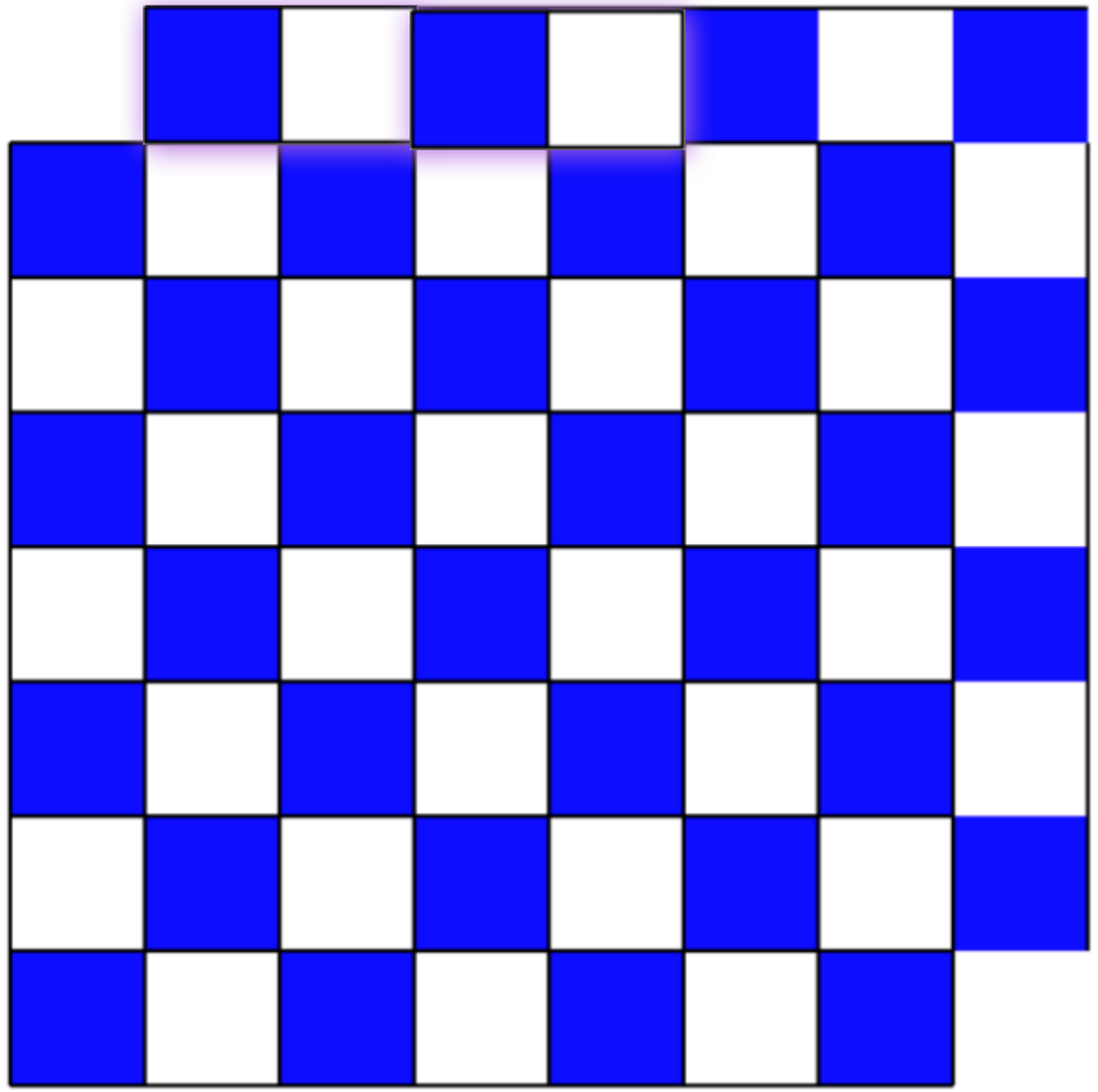




- Can we cover this board with 31 tiles of the form shown?
- Why can we quickly determine that the answer is no?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

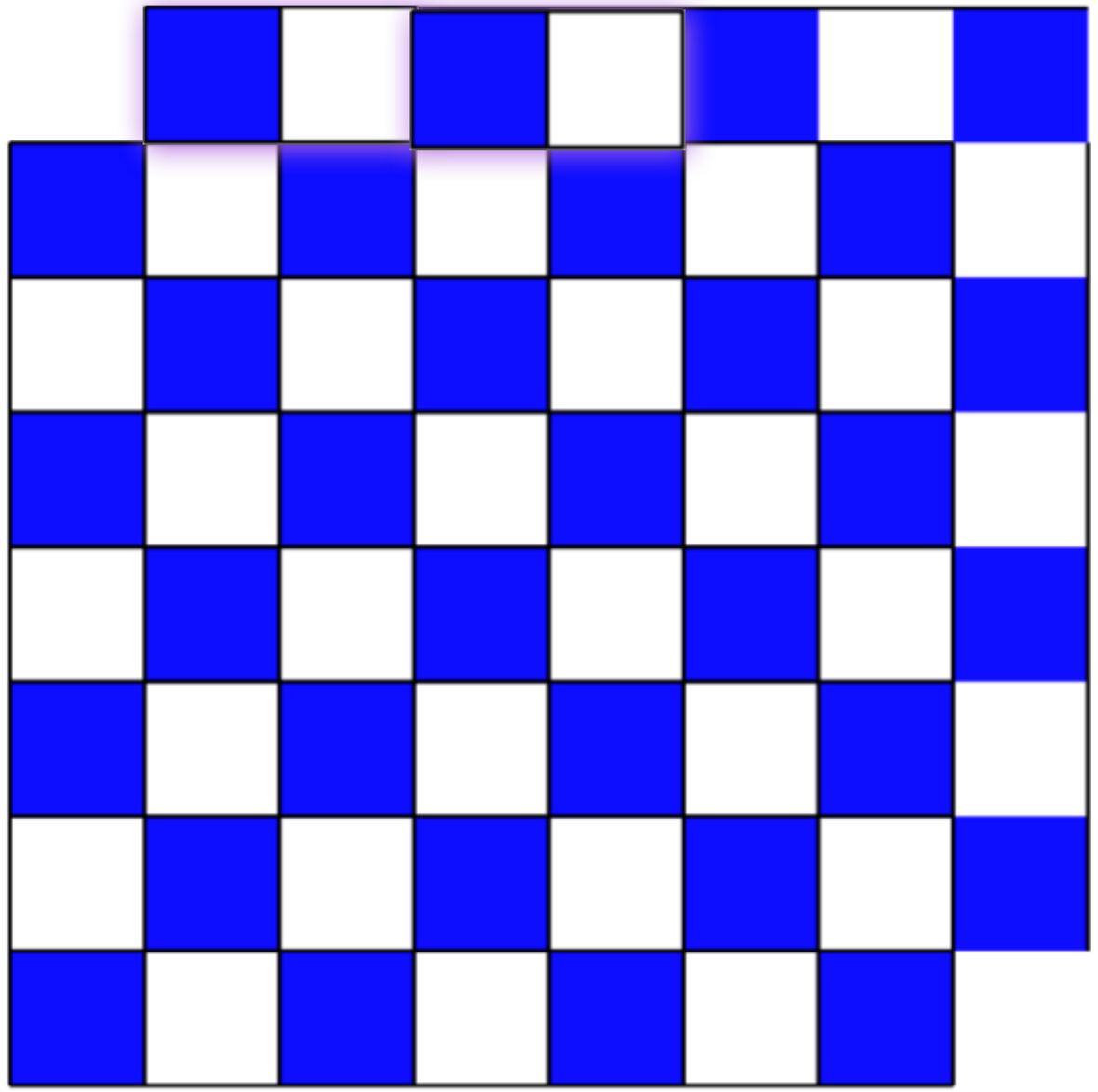
Transform and Conquer? Use abstraction?



- Can we cover this board with 31 tiles of the form shown?
- Why can we quickly determine that the answer is no?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Transform and Conquer? Use abstraction?



- Can we cover this board with 31 tiles of the form shown?
- Why can we quickly determine that the answer is no?
- **Hint:** Using the way the squares are coloured helps.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Algorithms and Data Structures



THE UNIVERSITY OF MELBOURNE

- **Algorithms**: for solving problems, transforming data.

- **Data structures**: for storing data; arranging data in a way that suits an algorithm.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Algorithms and Data Structures



THE UNIVERSITY OF MELBOURNE

- **Algorithms**: for solving problems, transforming data.

- **Data structures**: for storing data; arranging data in a way that suits an algorithm.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Algorithms and Data Structures



THE UNIVERSITY OF MELBOURNE

- **Algorithms**: for solving problems, transforming data.

- **Data structures**: for storing data; arranging data in a way that suits an algorithm.

- Which data structures are you familiar with?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Algorithms and Data Structures



THE UNIVERSITY OF MELBOURNE

- **Algorithms**: for solving problems, transforming data.

- **Data structures**: for storing data; arranging data in a way that suits an algorithm.
    - **Linear** data structures: stacks and queues
    - Trees and graphs
    - Dictionaries
- Which data structures are you familiar with?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Array



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Array

- An array corresponds to a sequence of consecutive cells in memory.
- Depending on programming language: `A[0]` up to `A[n-1]`, or `A[1]` up to `A[n]`.
- Locating a cell, and storing or retrieving data at that cell is very fast.



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Array in Memory

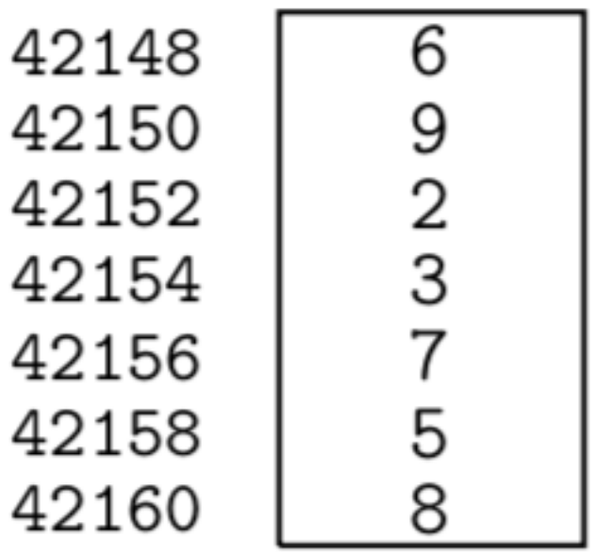


42148	6
42150	9
42152	2
42154	3
42156	7
42158	5
42160	8

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Array in Memory





How many bytes does each integer occupy here?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

The Array in Memory



42148	6
42150	9
42152	2
42154	3
42156	7
42158	5
42160	8

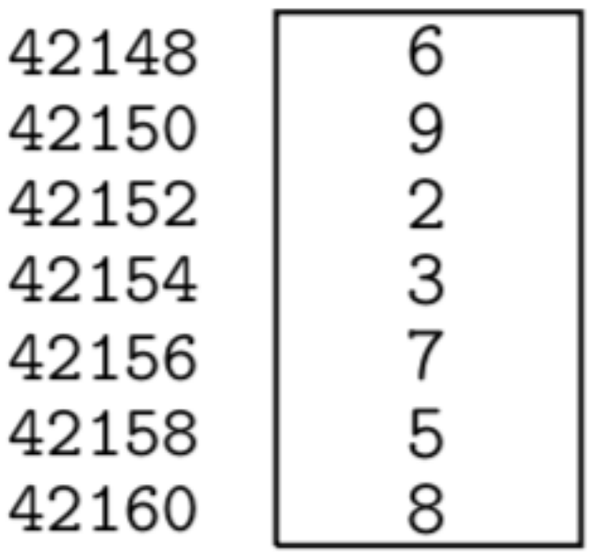
How many bytes does each integer occupy here?

Answer: 2 (16-bit integers)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Array

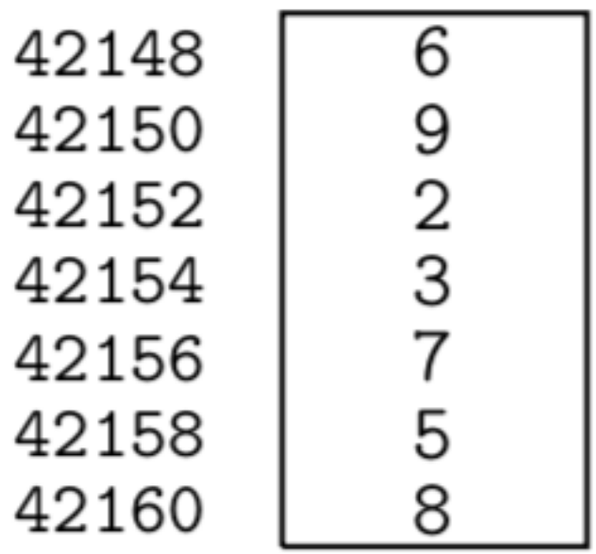




Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Array





The downside of an array is that maintaining a contiguous bank of cells with information can be difficult and time-consuming.

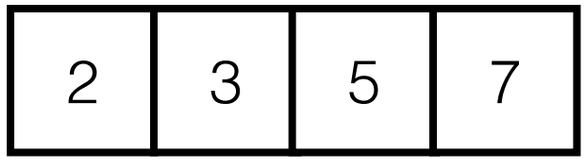
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List



THE UNIVERSITY OF MELBOURNE

An array **x**:



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List



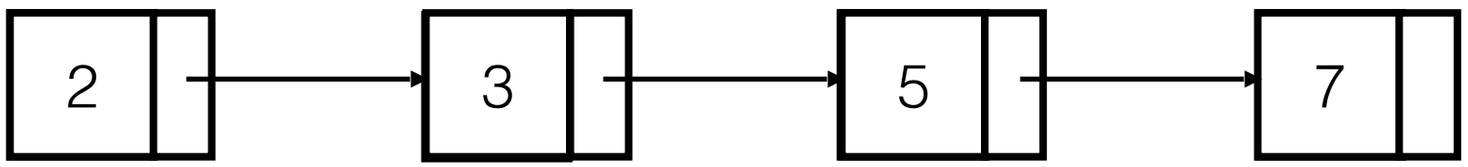
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List



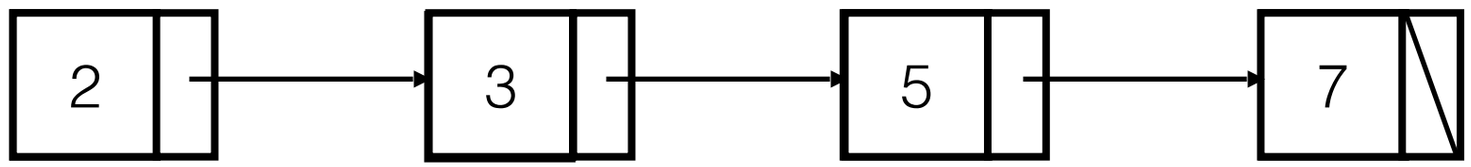
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List



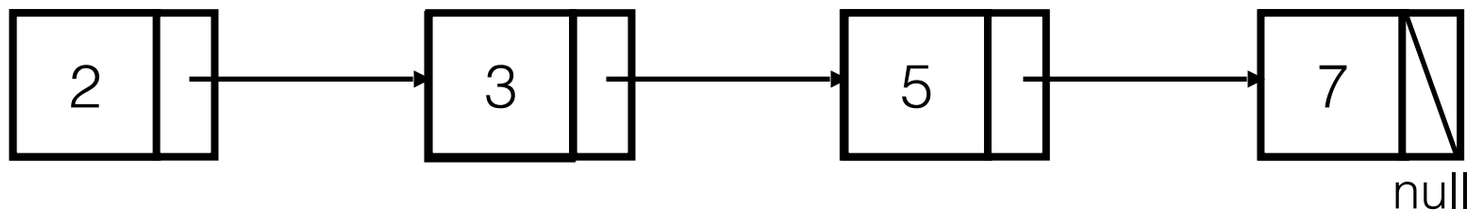
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List



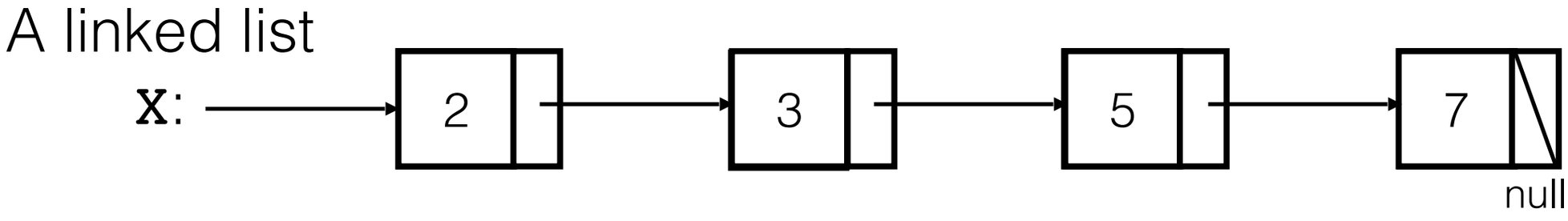
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List



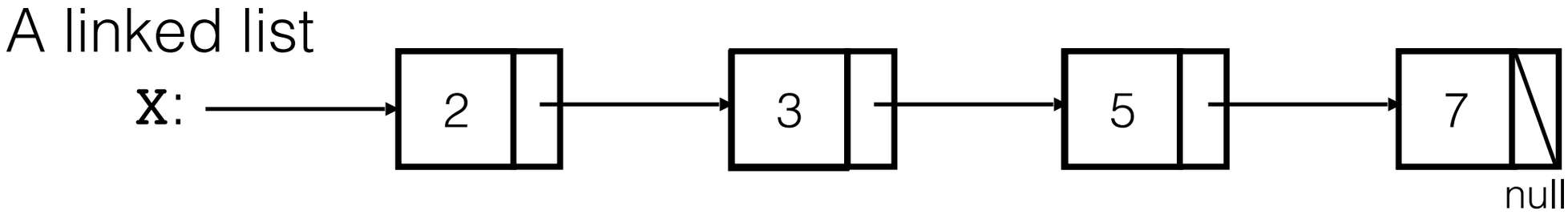
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

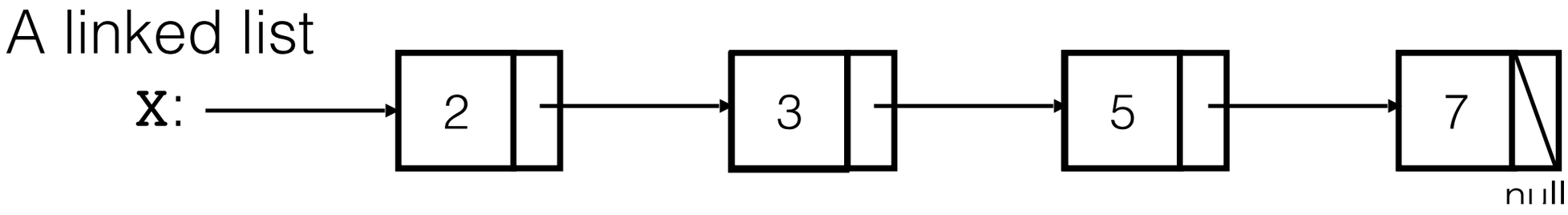
Primitive Data Structures: The Linked List



Suppose variable `x` holds the address 42160, then the list could look like this in memory:

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List

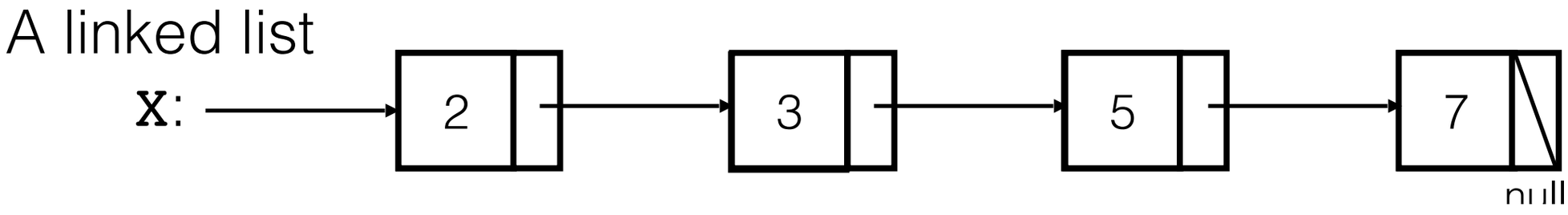


Suppose variable `x` holds the address 42160, then the list could look like this in memory:

42148	3
42150	42152
42152	5
42154	42164
42156	
42158	
42160	2
42162	42148
42164	7
42166	0

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List

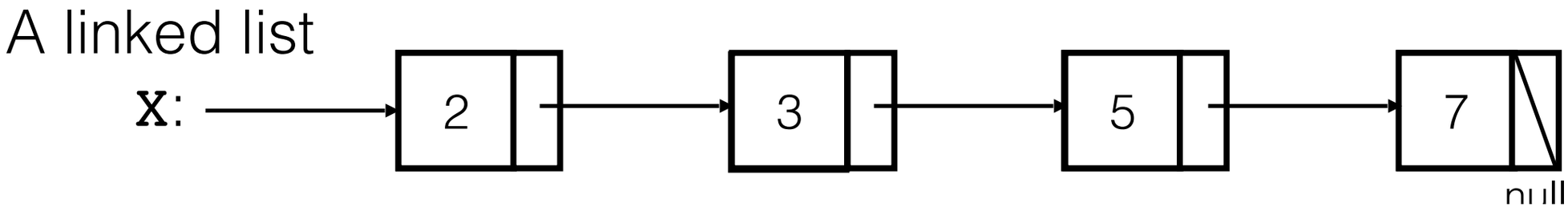


Suppose variable `x` holds the address 42160, then the list could look like this in memory:

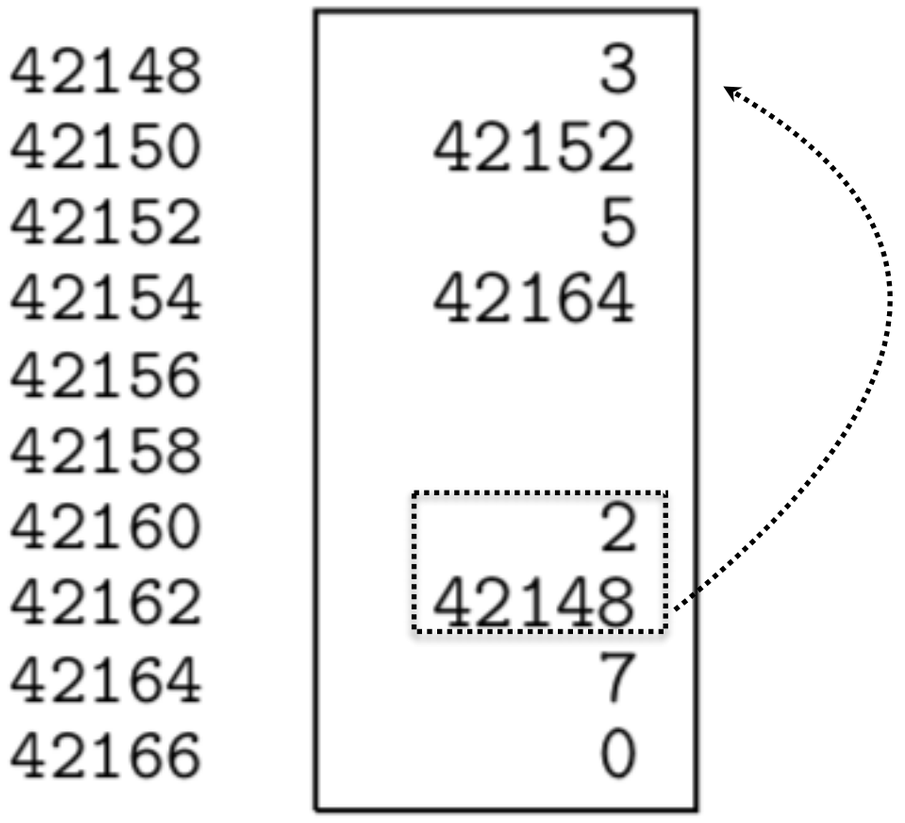
42148	3
42150	42152
42152	5
42154	42164
42156	
42158	
42160	2
42162	42148
42164	7
42166	0

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List

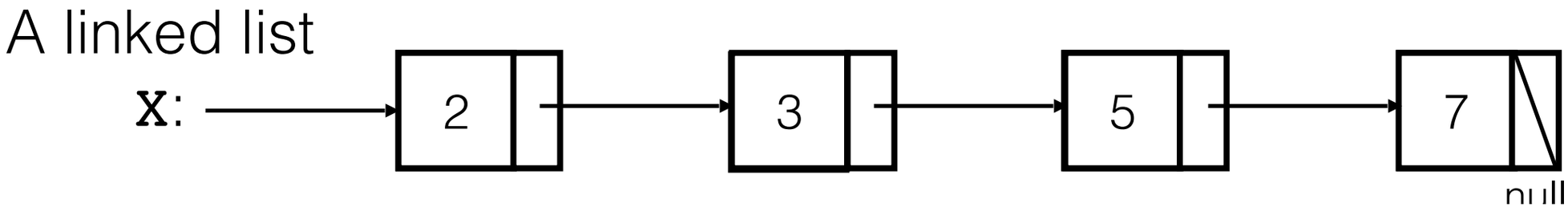


Suppose variable `x` holds the address 42160, then the list could look like this in memory:

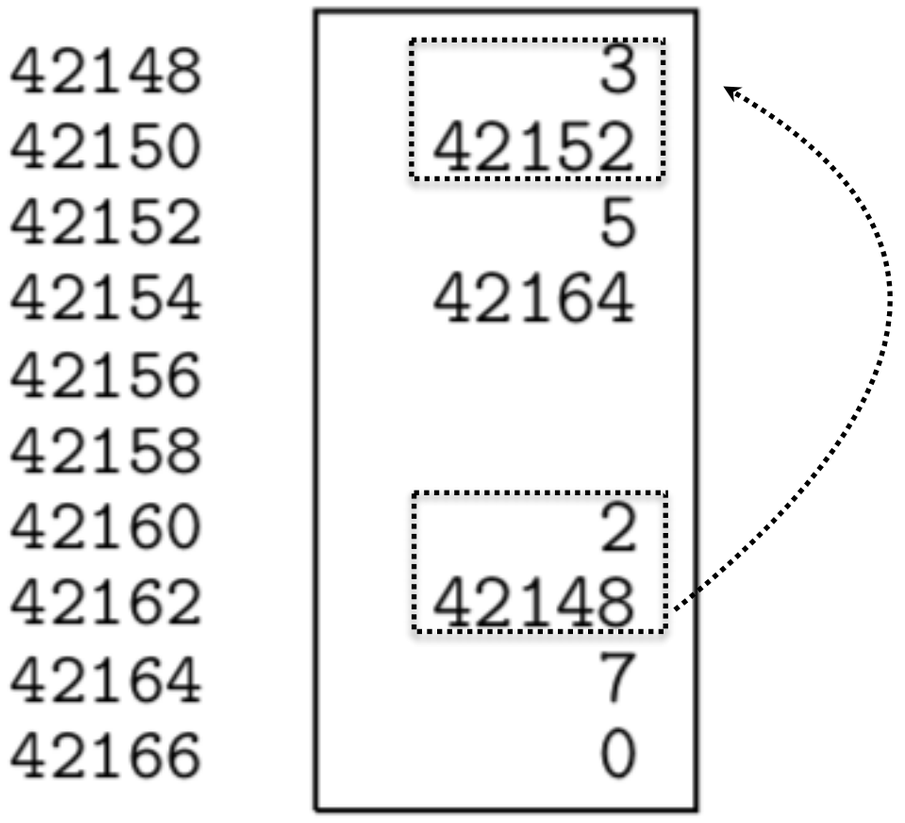


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List

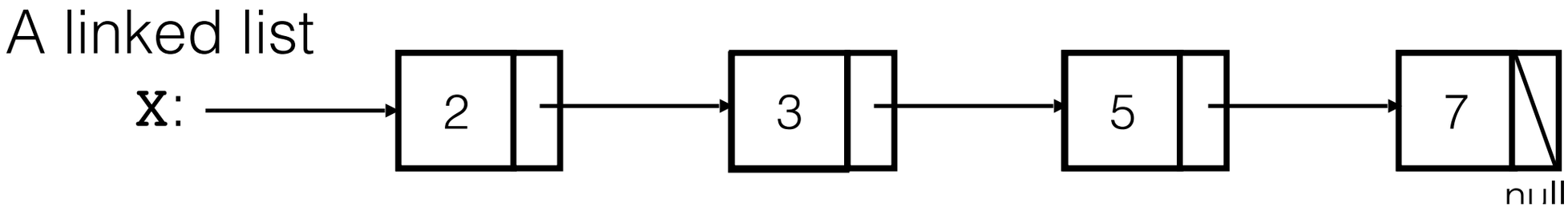


Suppose variable `x` holds the address 42160, then the list could look like this in memory:

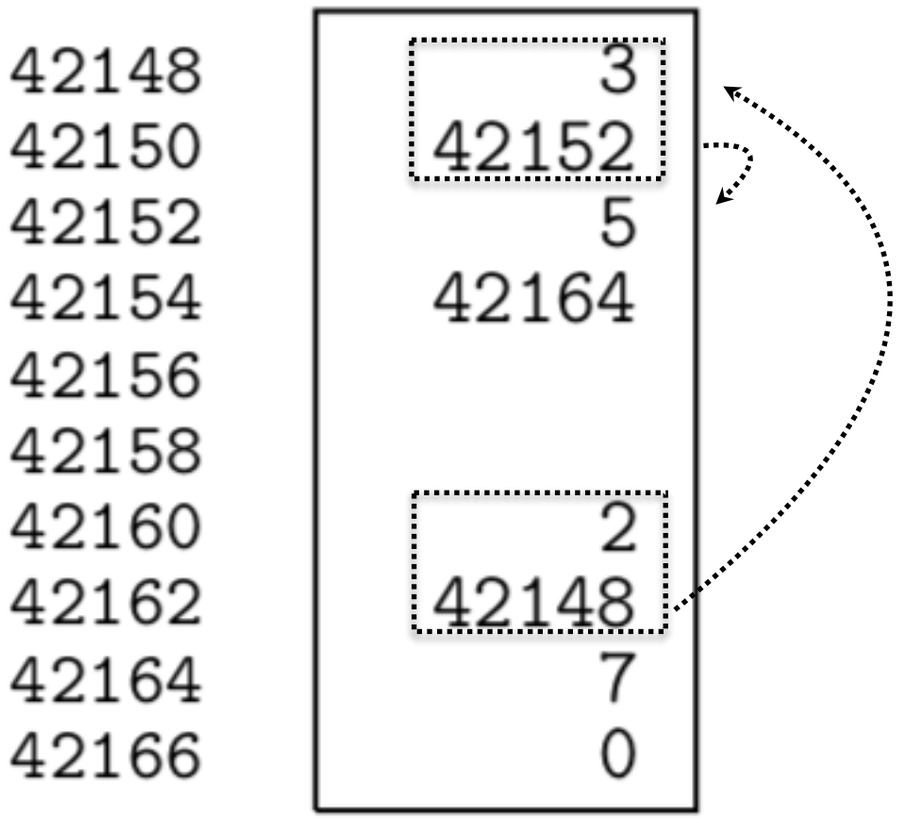


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List

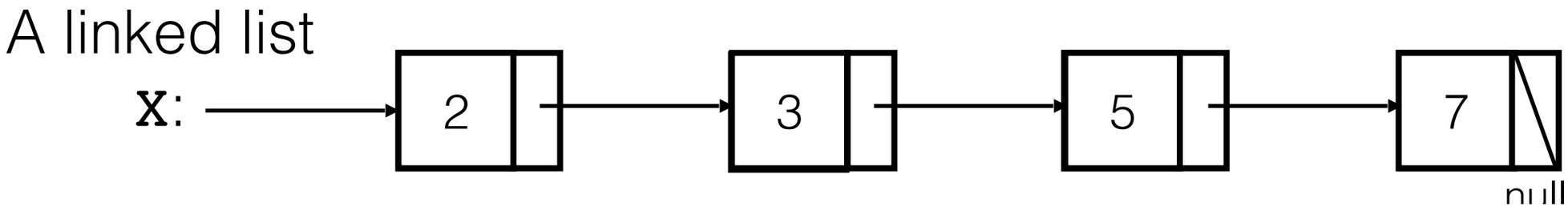


Suppose variable `x` holds the address 42160, then the list could look like this in memory:

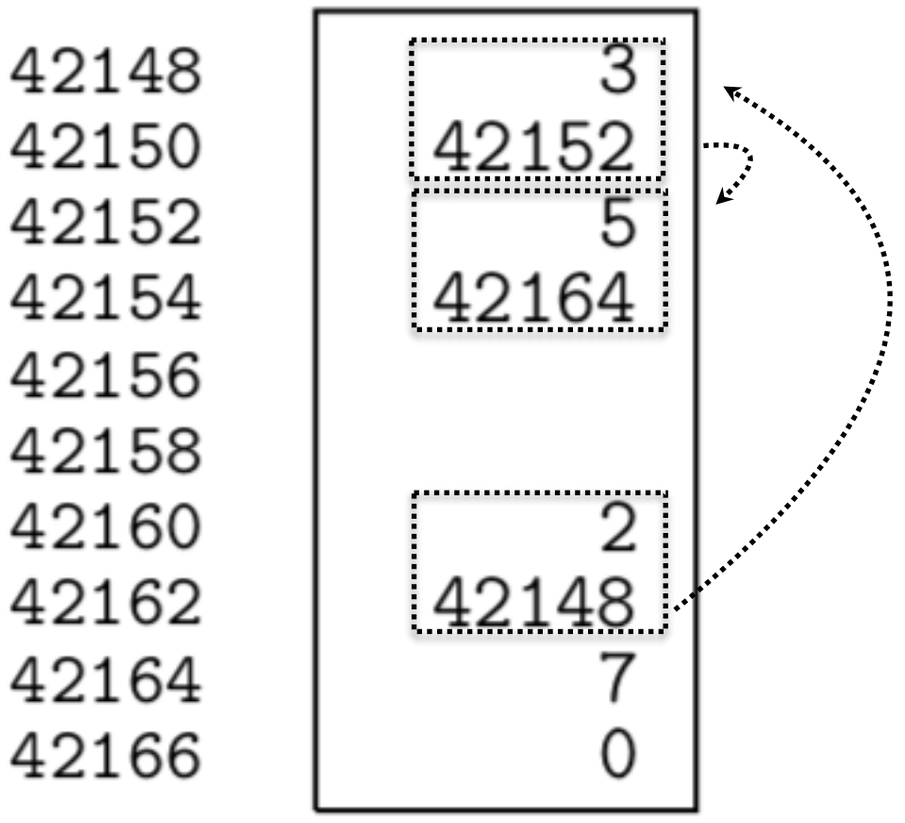


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List

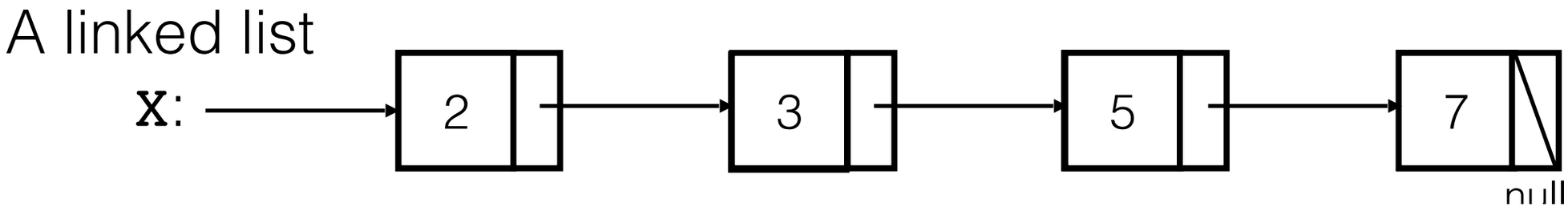


Suppose variable `x` holds the address 42160, then the list could look like this in memory:

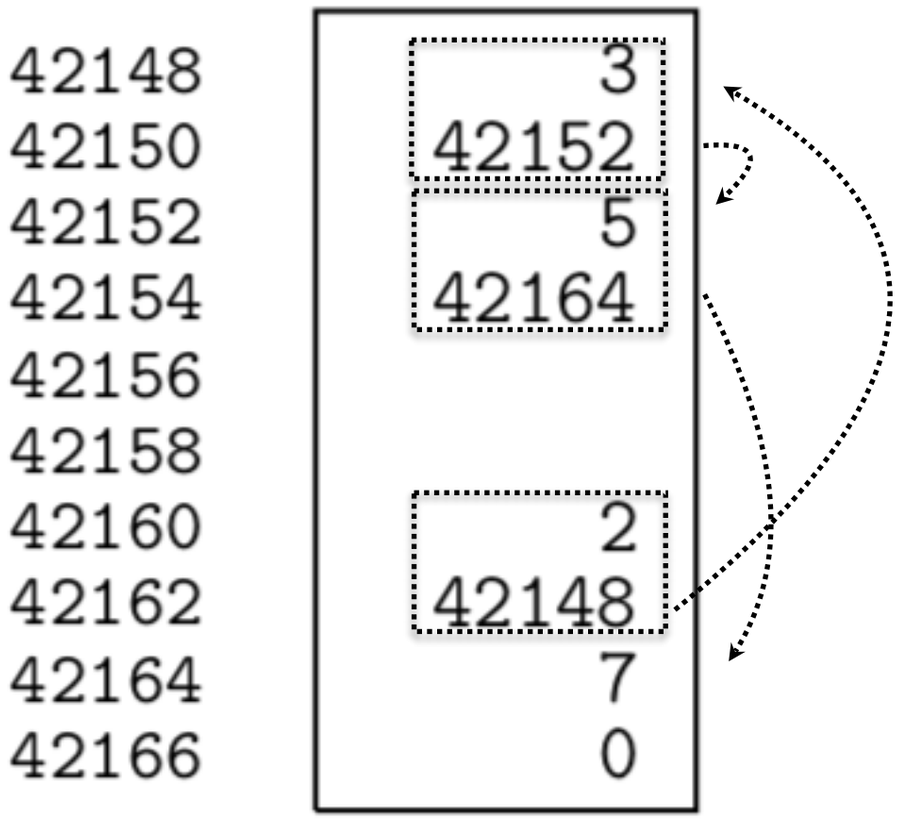


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List

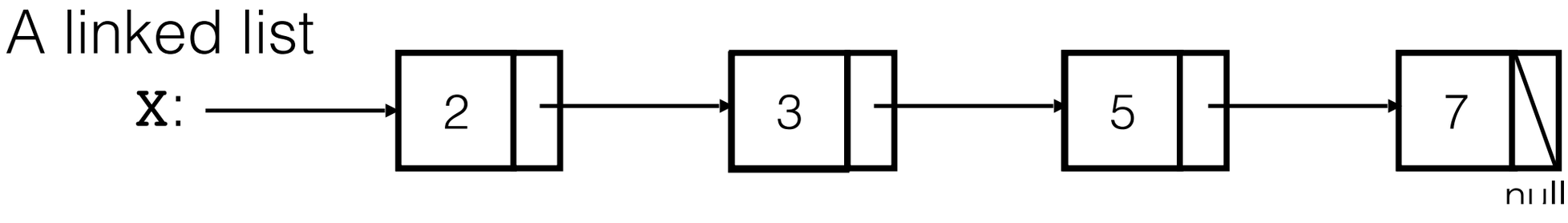


Suppose variable `x` holds the address 42160, then the list could look like this in memory:

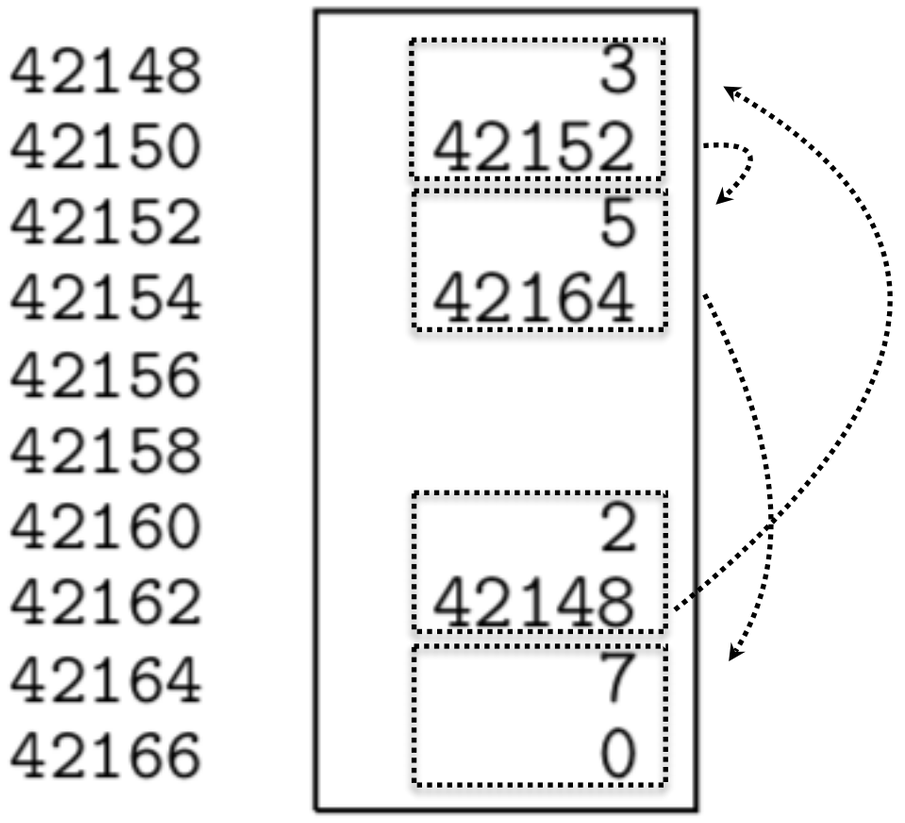


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List

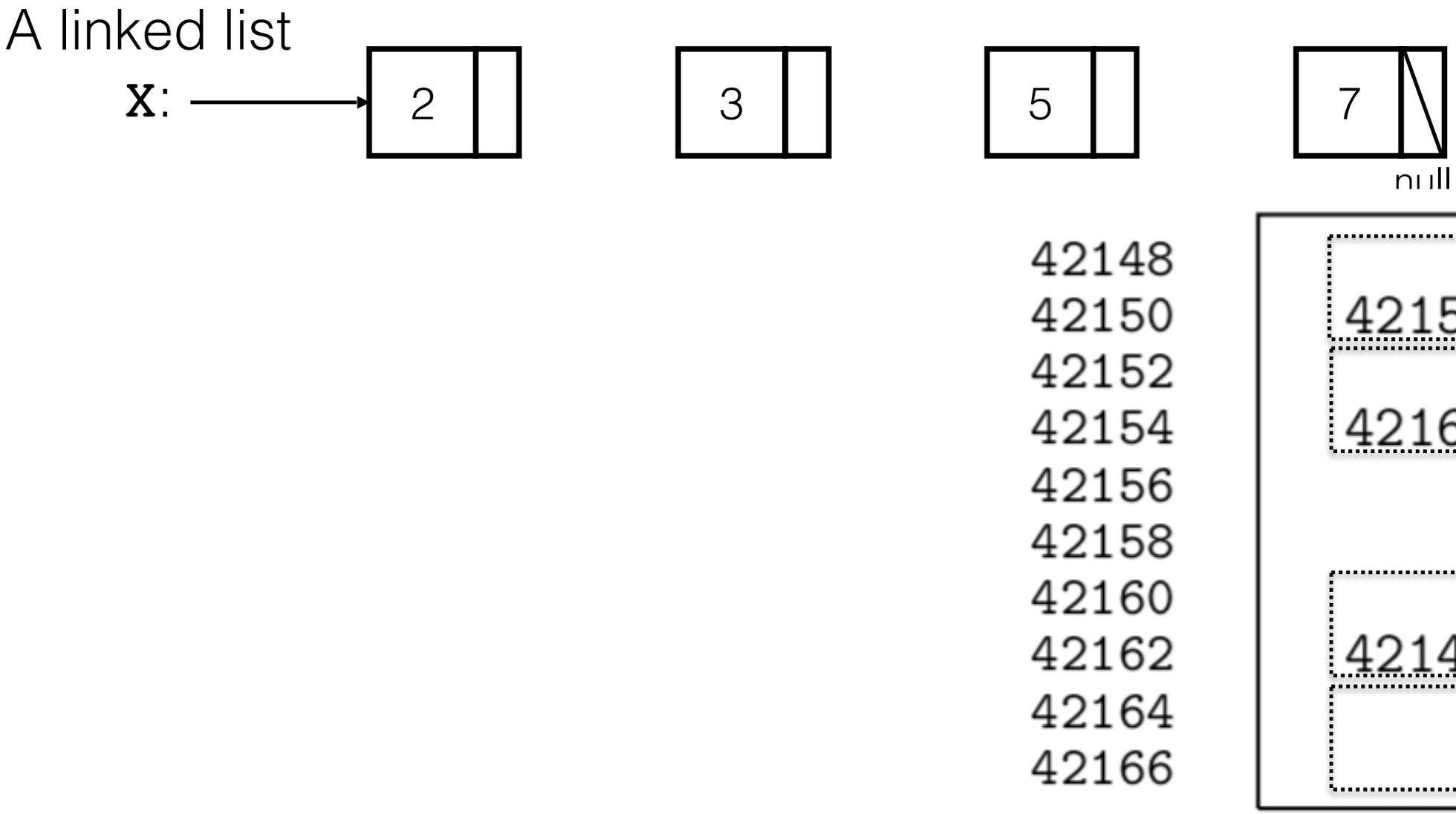


Suppose variable `x` holds the address 42160, then the list could look like this in memory:



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Primitive Data Structures: The Linked List

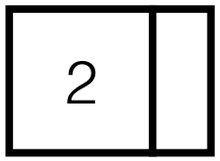


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Terminology

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

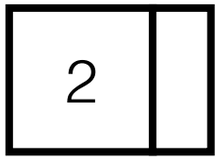
Terminology



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

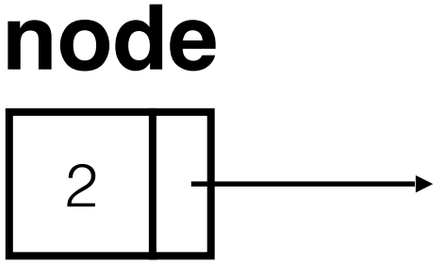
Terminology

node



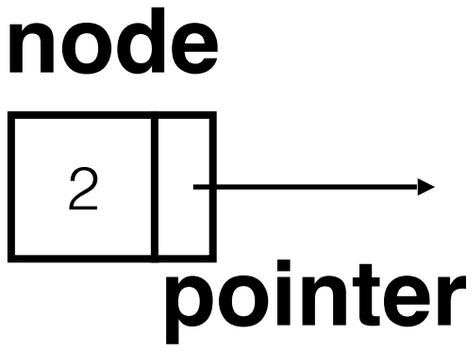
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Terminology



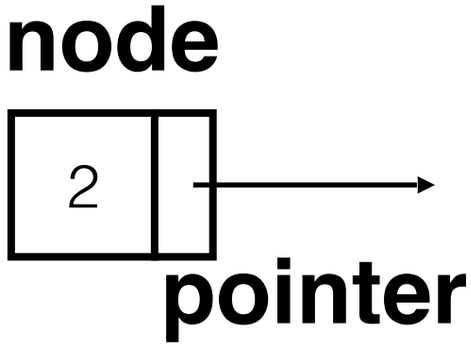
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Terminology



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

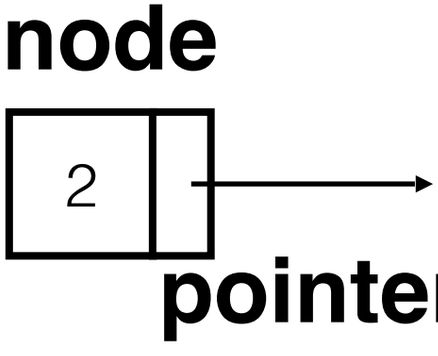
Terminology



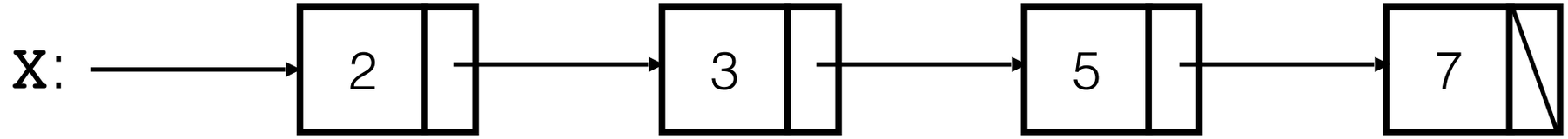
(in Java: “reference”)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Terminology

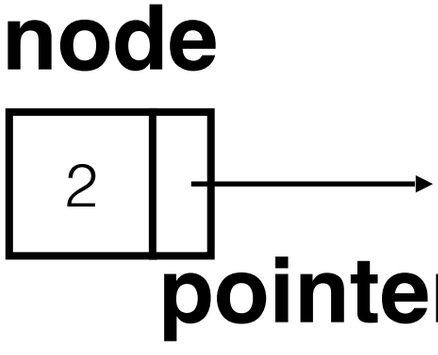


(in Java: “reference”)

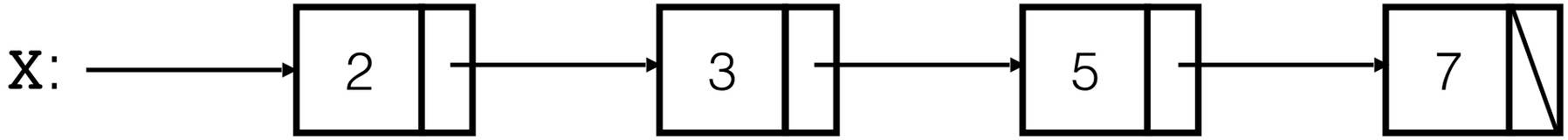


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Terminology



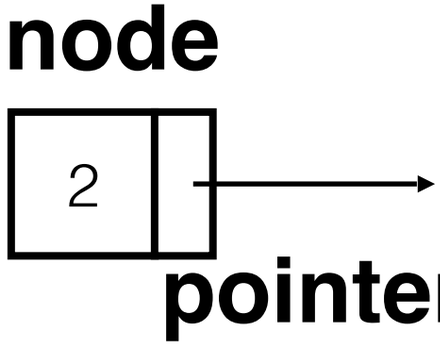
(in Java: “reference”)



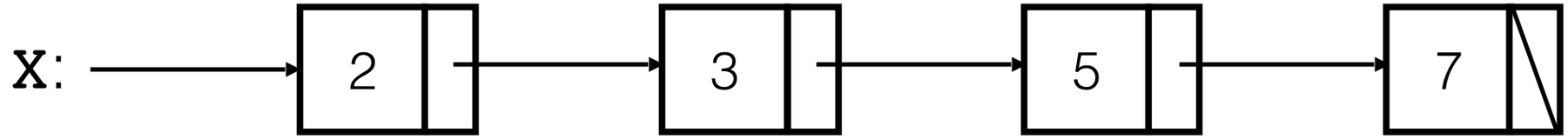
`x` is (a pointer to) the **head node** of the list

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

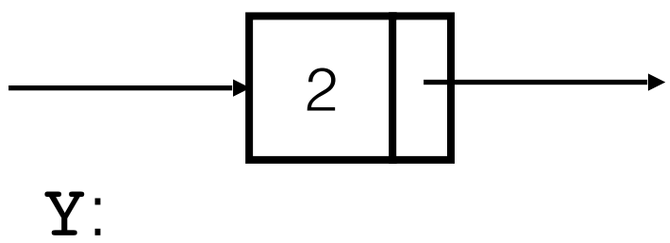
Terminology



(in Java: “reference”)

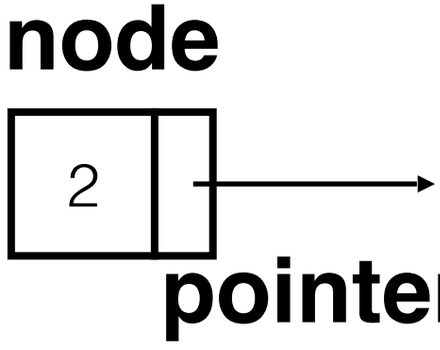


`x` is (a pointer to) the **head node** of the list

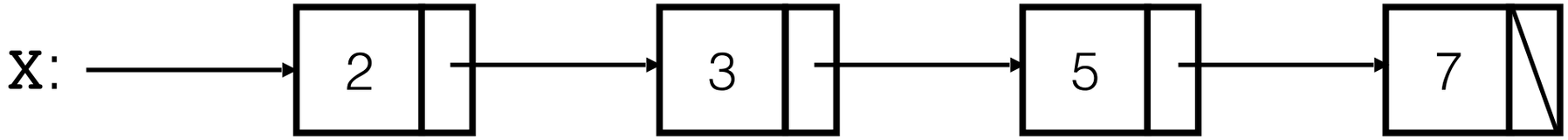


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

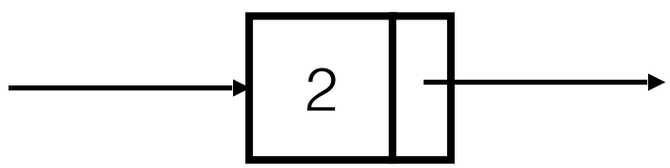
Terminology



(in Java: “reference”)



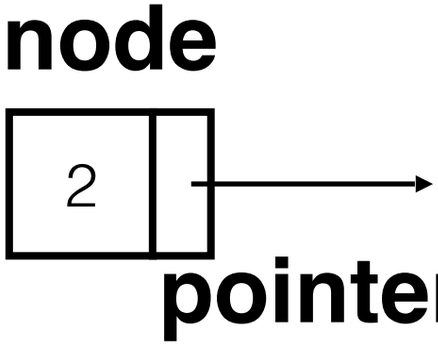
`x` is (a pointer to) the **head node** of the list



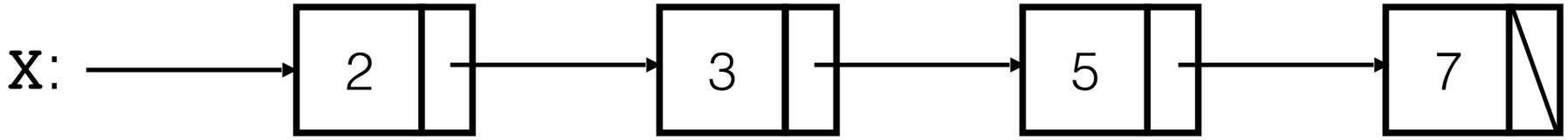
“`Y.val`” *refers to*

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

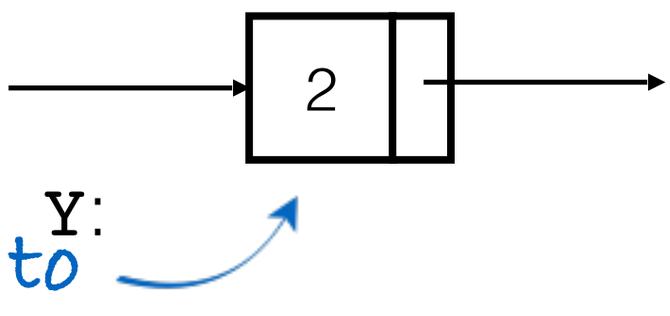
Terminology



(in Java: “reference”)



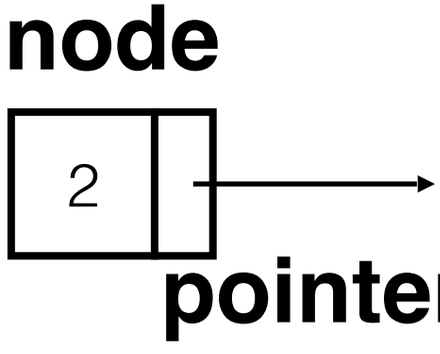
`x` is (a pointer to) the **head node** of the list



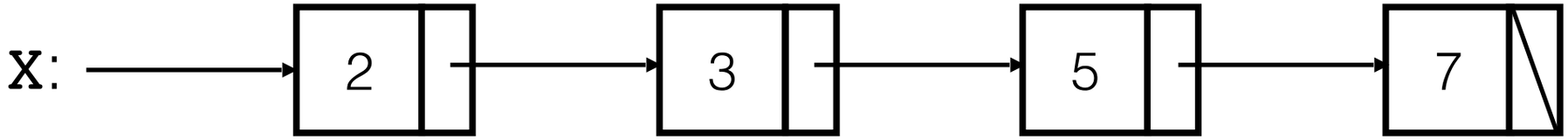
“`Y.val`” *refers to*

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

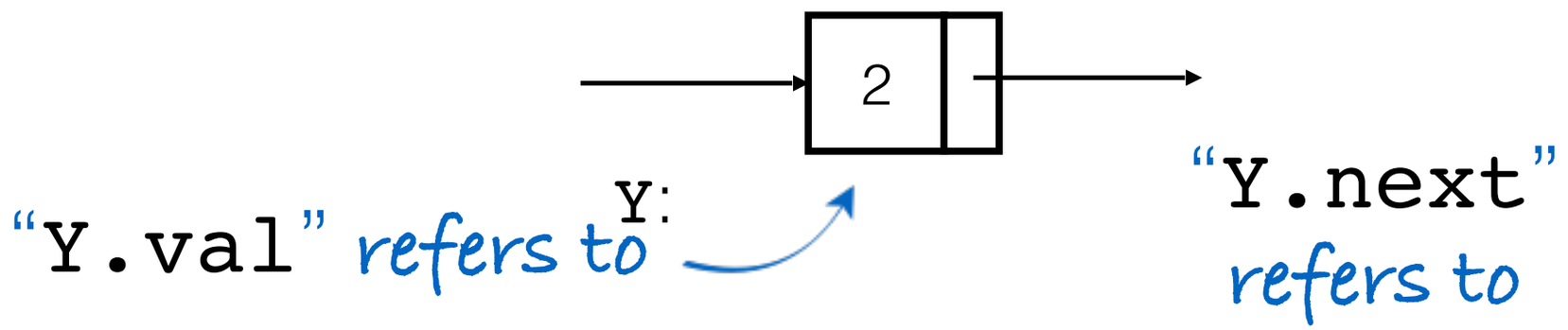
Terminology



(in Java: “reference”)

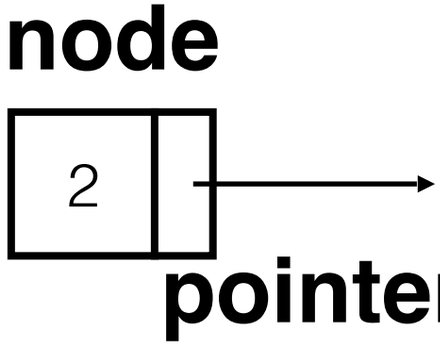


`x` is (a pointer to) the **head node** of the list

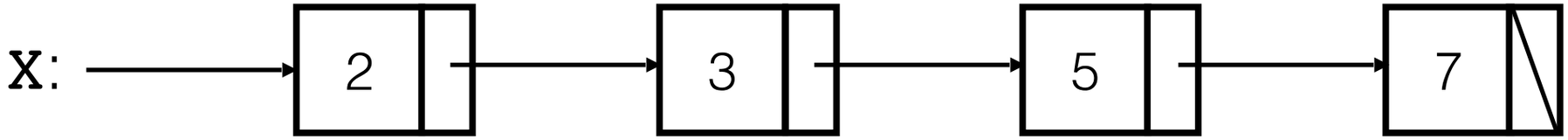


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

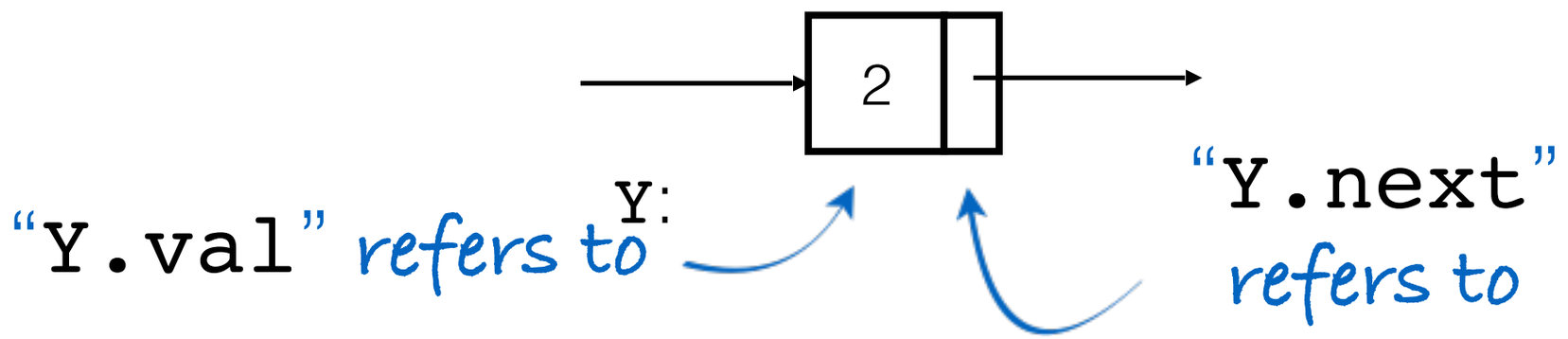
Terminology



(in Java: “reference”)



`x` is (a pointer to) the **head node** of the list



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Linked List

- Often we use a dummy head node that points to the first object, or to a special `null` object that represents an empty list. This makes it easier to write functions that insert or delete elements.

- Inserting and deleting elements is very fast: just move a few links around.

- Finding the *i*th element can be time-consuming.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

Y:



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

Y:



Let's trace the execution of **find(Y,7,7)**

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

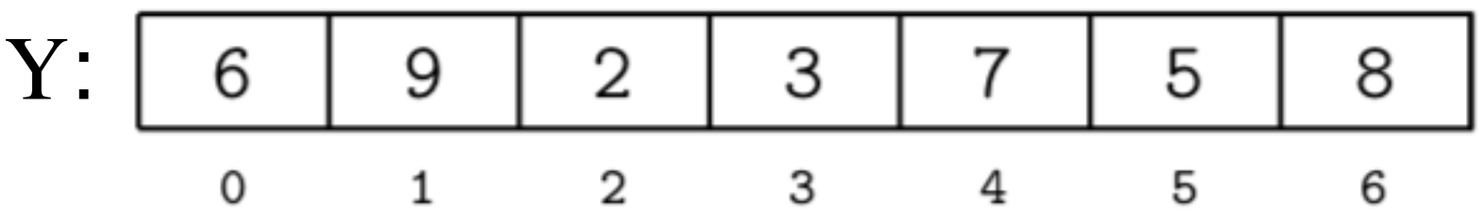
- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

A: Y

Y:



Let's trace the execution of **find(Y,7,7)**

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

A: Y x: 7

Y:



Let's trace the execution of find(Y,7,7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

A: Y x: 7 n: 7

Y:



Let's trace the execution of find(Y,7,7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

- Walk through the array (of length n)
- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

A: Y x: 7 n: 7 j: 0

Y:

6	9	2	3	7	5	8
0	1	2	3	4	5	6

Let's trace the execution of find(Y,7,7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

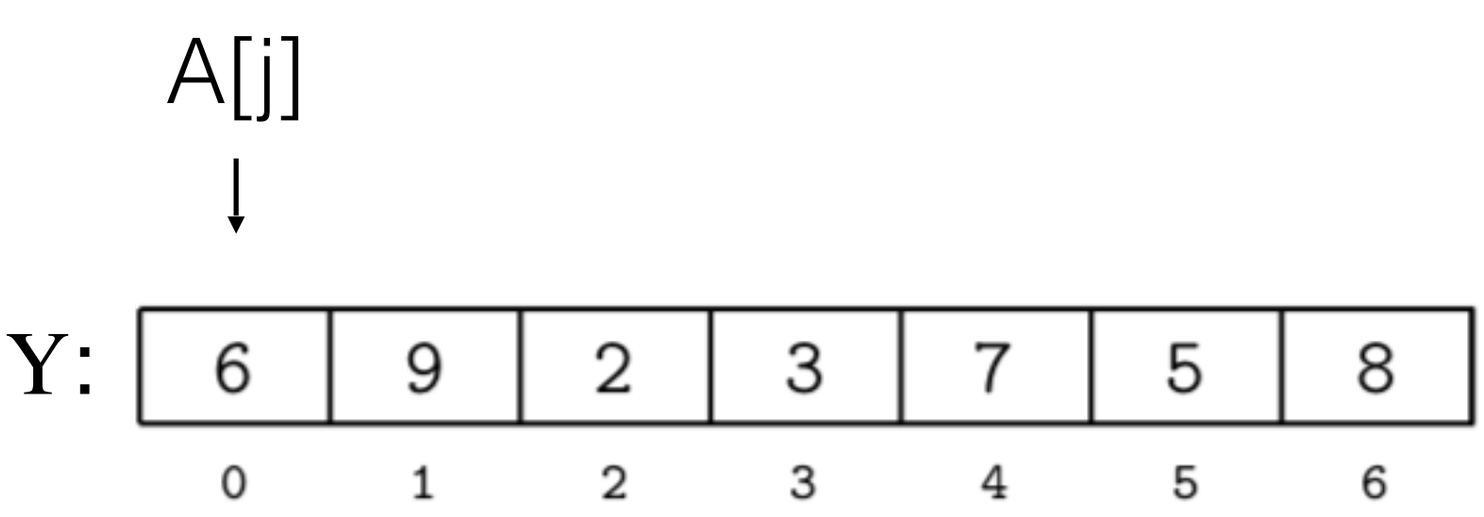
Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

A: Y x: 7 n: 7 j: 0



Let's trace the execution of find(Y,7,7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

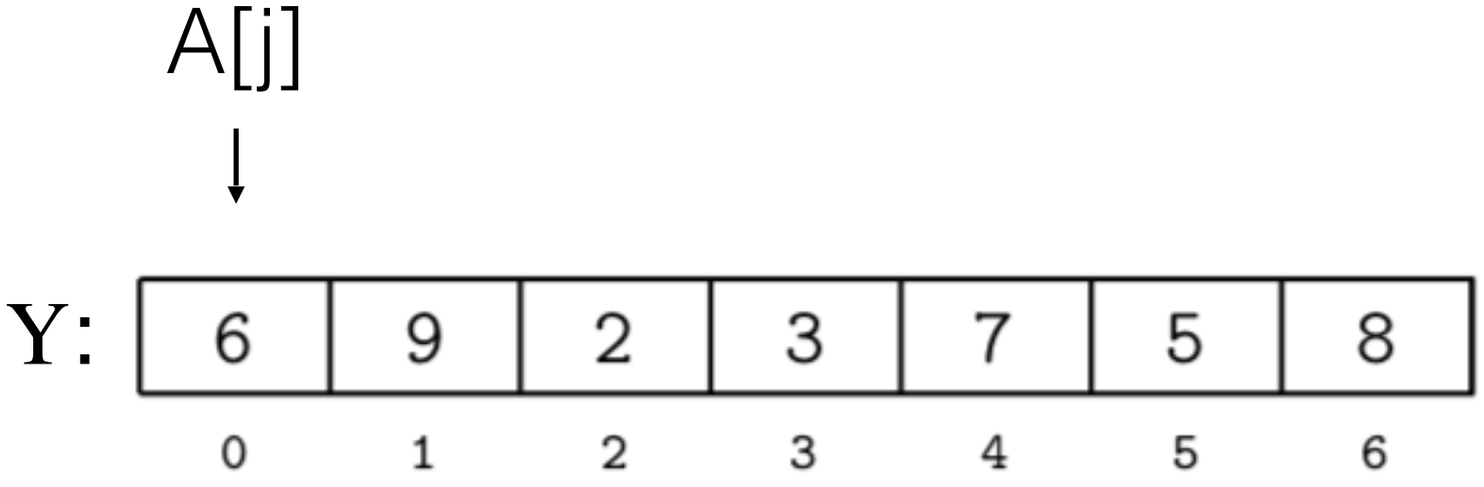
Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

A: Y x: 7 n: 7 j: 1



Let's trace the execution of find(Y,7,7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

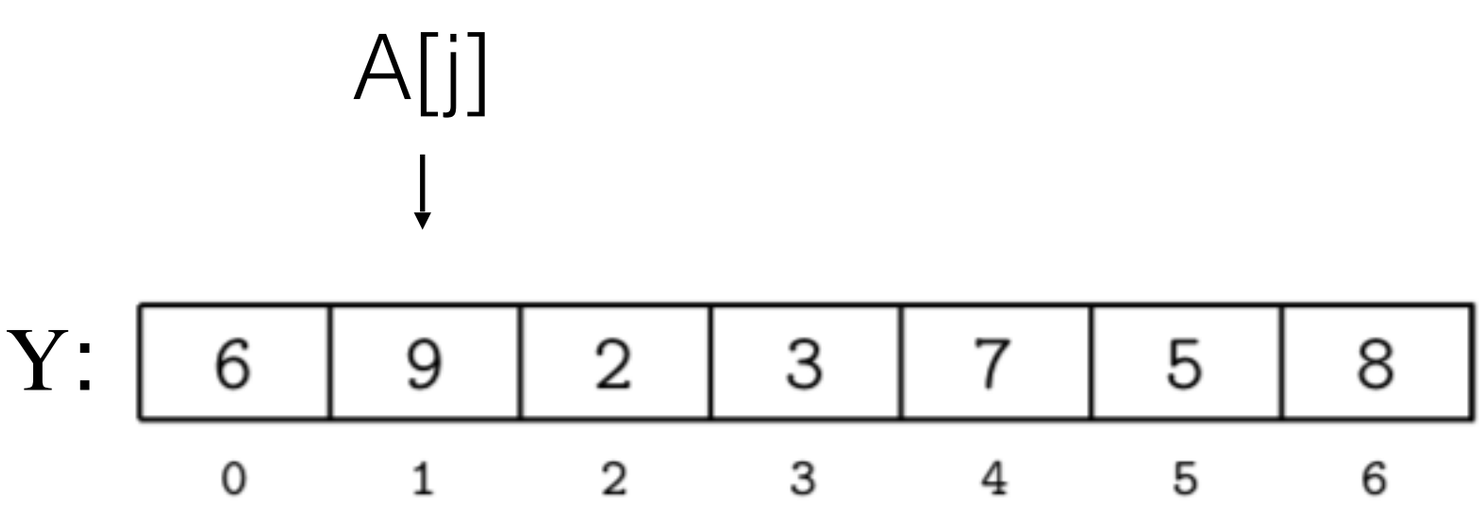
Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

A: Y x: 7 n: 7 j: 1



Let's trace the execution of find(Y,7,7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

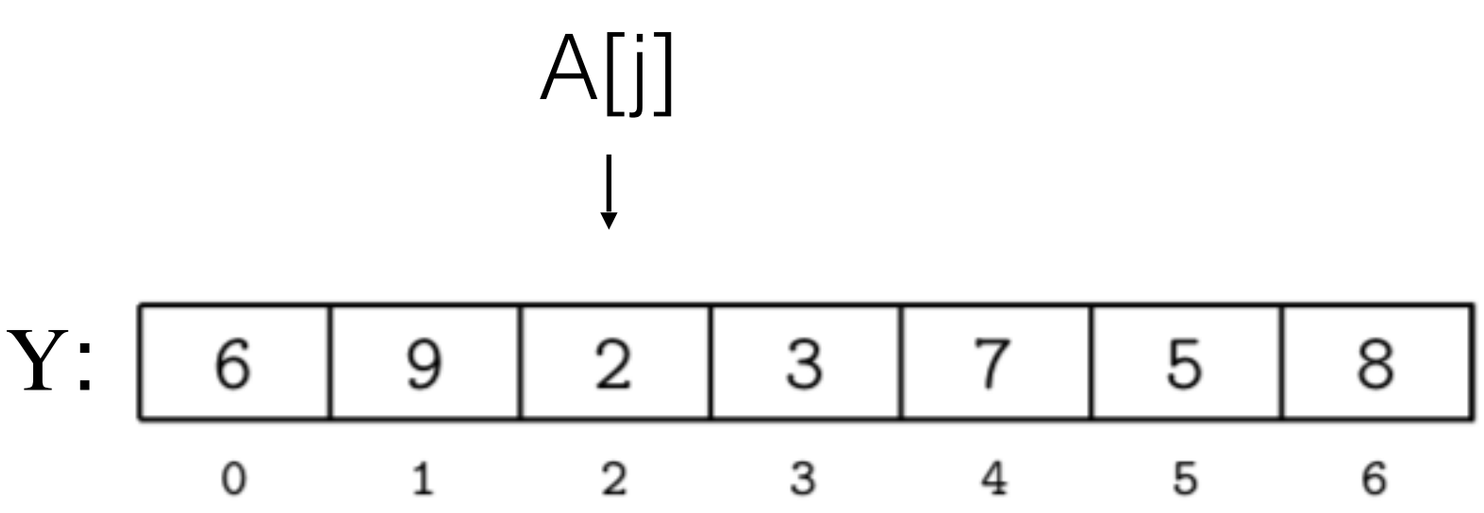
Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

A: Y x: 7 n: 7 j: 2



Let's trace the execution of find(Y,7,7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

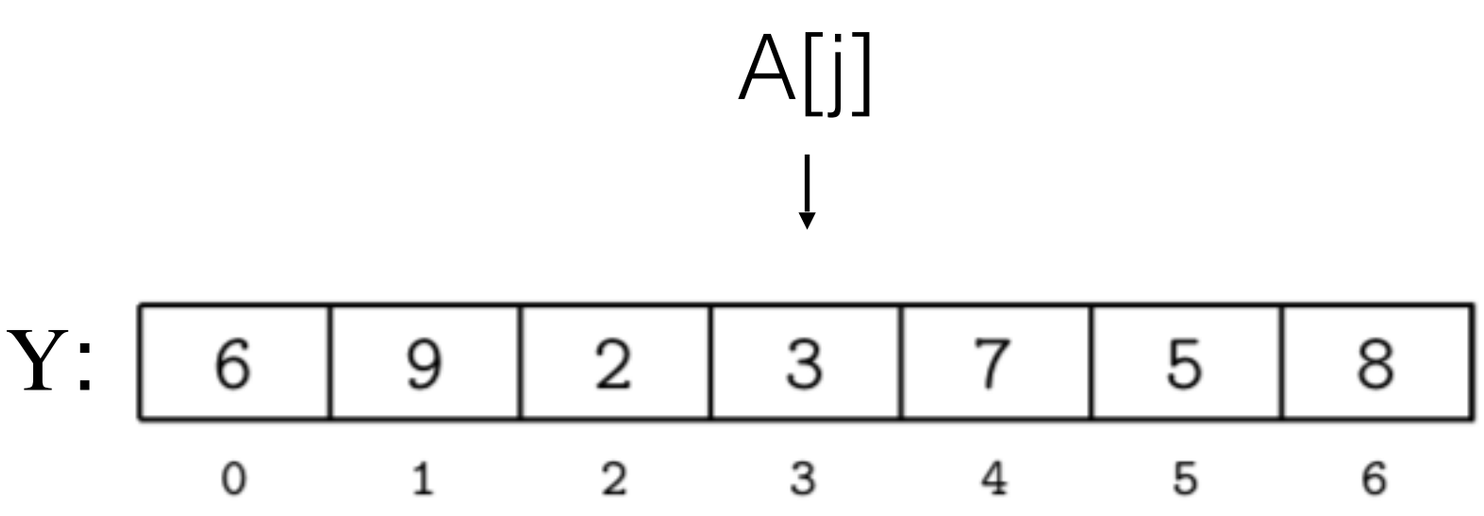
Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)
    j ← 0
    while j < n
        if A[j] = x
            return j
        j ← j+1
    return -1
```

A: Y x: 7 n: 7 j: 3



Let's trace the execution of find(Y,7,7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)

    j ← 0

    while j < n

        if A[j] = x

            return j

        j ← j+1

    return -1
```

A: Y x: 7 n: 7 j: 4

A[j]



Let's trace the execution of find(Y,7,7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: Array

- Walk through the array (of length n)

- For example, to locate item **x**.

```
function find(A,x,n)

    j ← 0

    while j < n

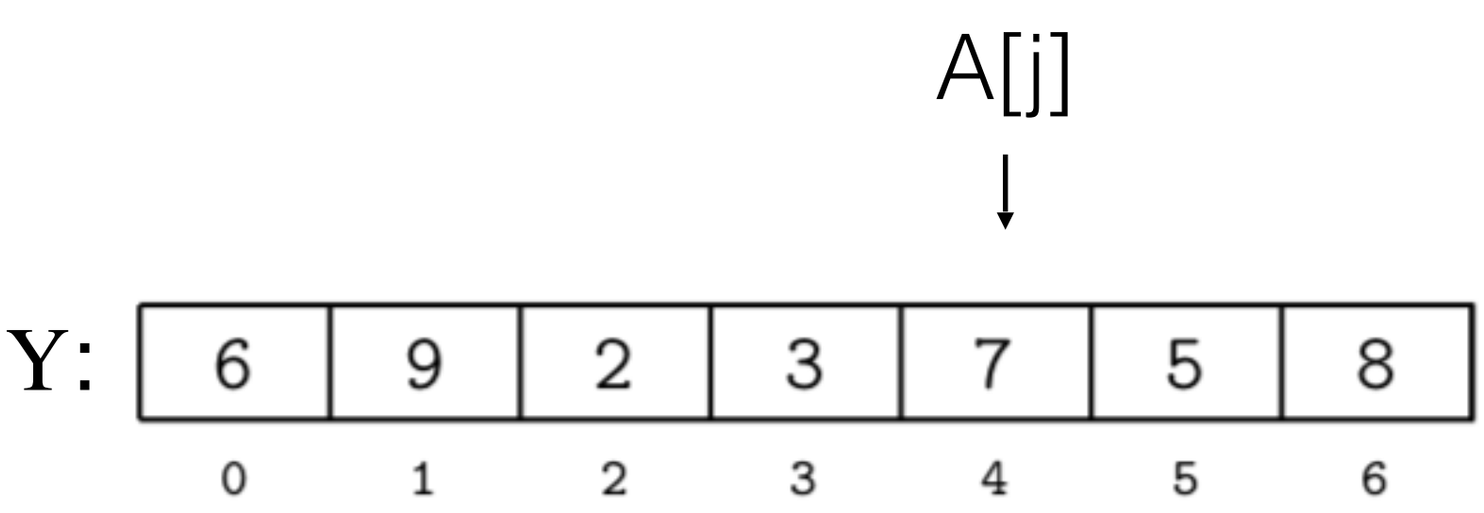
        if A[j] = x

            return j

        j ← j+1

    return -1
```

A: Y x: 7 n: 7 j: 4



Let's trace the execution of find(Y,7,7)

(returns 4)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item **x**.

```
function find(head,x)
    p ← head
    while p ≠ null
      if p.val = x
        return p
      p ← p.next
  return null
```

```
function find(A,x,n)
    j ← 0
    while j < n
      if A[j] = x
        return j
      j ← j+1
  return -1
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item **x**.

(note similarity to array version)

```
function find(head,x)
    p ← head
    while p ≠ null
      if p.val = x
        return p
      p ← p.next
  return null
```

```
function find(A,x,n)
    j ← 0
    while j < n
      if A[j] = x
        return j
      j ← j+1
  return -1
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List



THE UNIVERSITY OF MELBOURNE

- Walk through a linked list.
- For example, to locate item `x`.

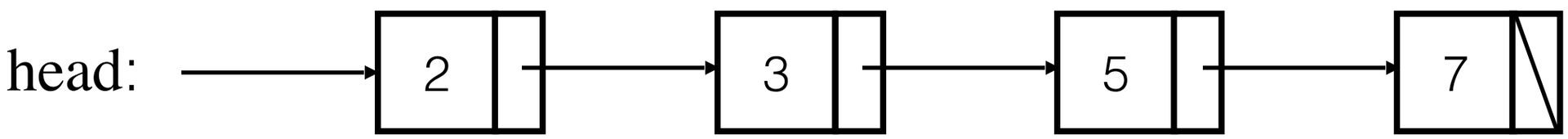
```
function find(head,x)
    p ← head
    while p ≠ null
      if p.val = x
        return p
      p ← p.next
  return null
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item `x`.

```
function find(head,x)
    p ← head
    while p ≠ null
      if p.val = x
        return p
      p ← p.next
  return null
```

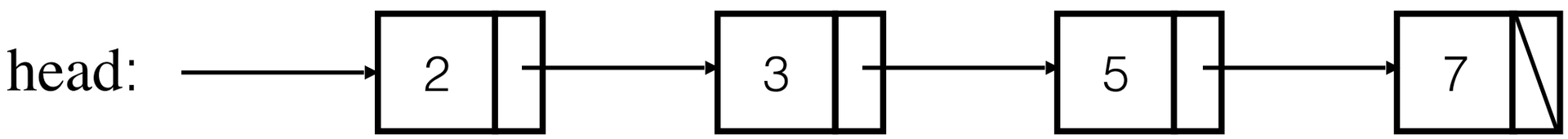


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item `x`.

```
function find(head,x)
    p ← head
    while p ≠ null
        if p.val = x
            return p
        p ← p.next
    return null
```



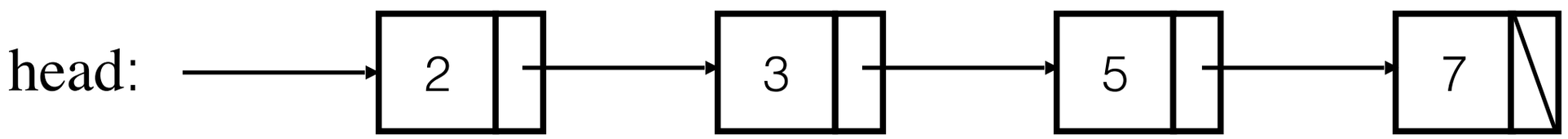
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item `x`.

```
function find(head,x)
    p ← head
    while p ≠ null
      if p.val = x
          return p
      p ← p.next
  return null
```

p:

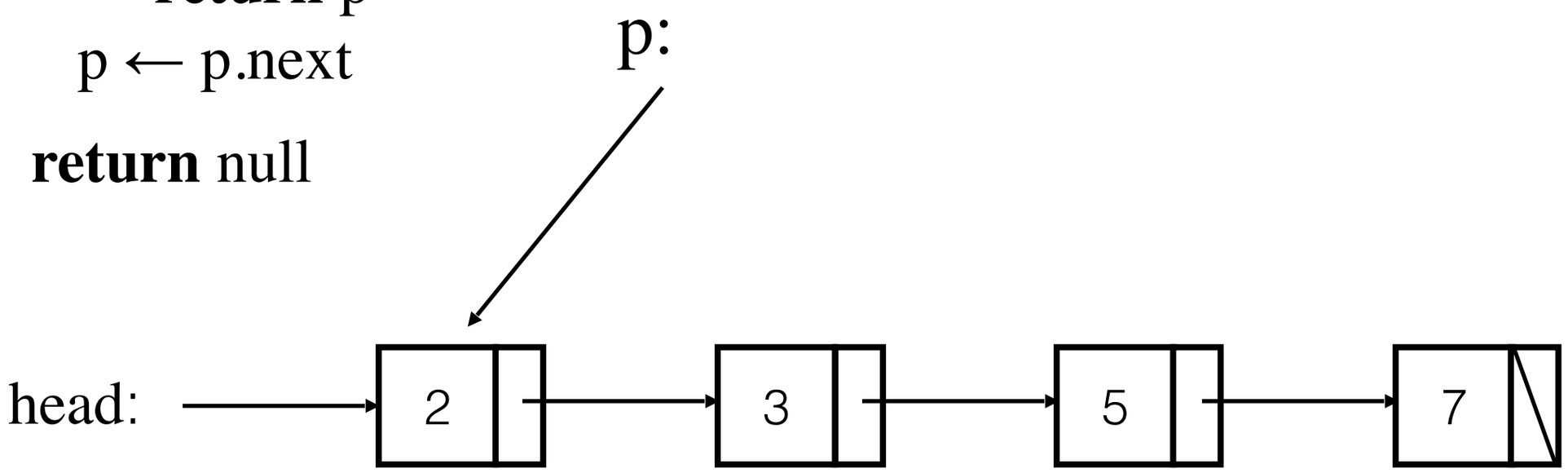


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item `x`.

```
function find(head,x)
    p ← head
    while p ≠ null
        if p.val = x
            return p
        p ← p.next
    return null
```

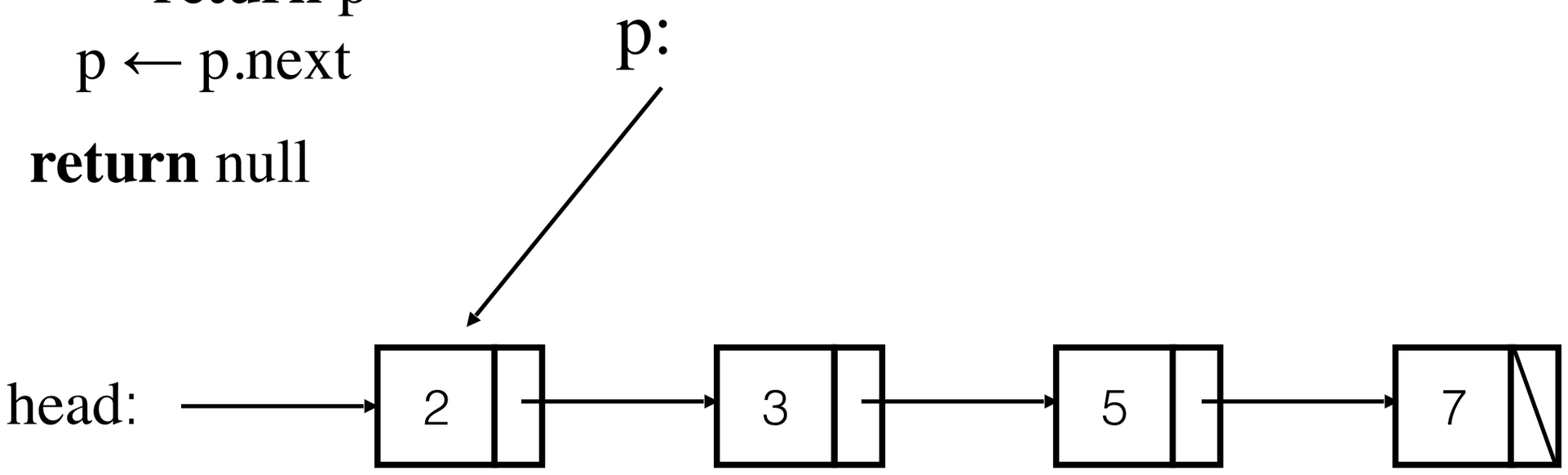


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item `x`.

```
function find(head,x)
    p ← head
    while p ≠ null
      if p.val = x
        return p
    p ← p.next
  return null
```

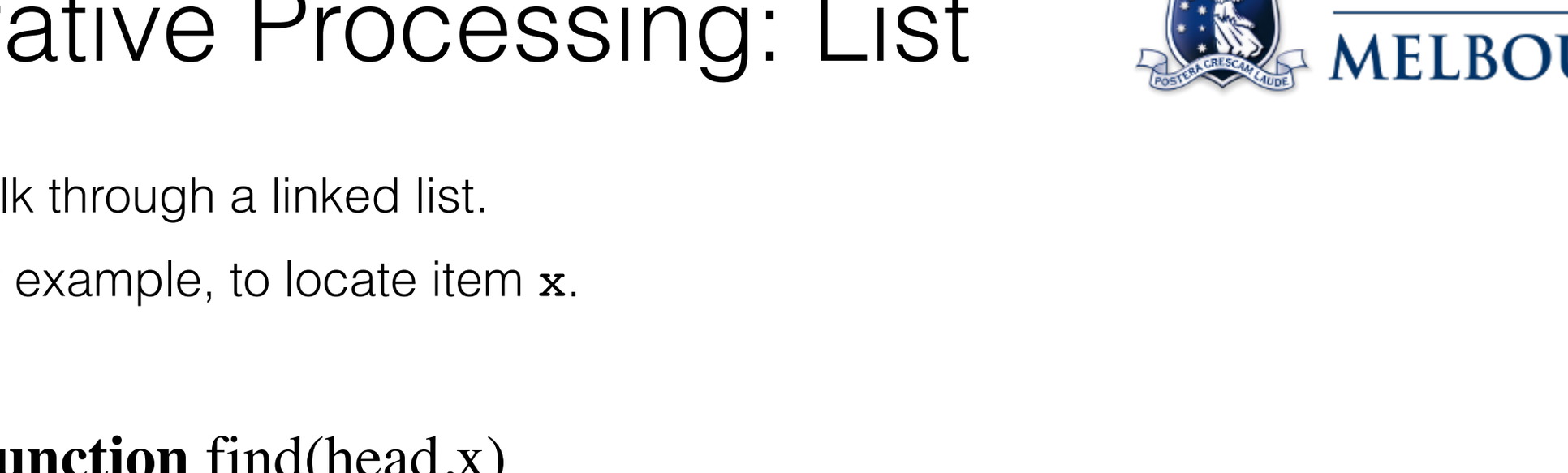


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item `x`.

```
function find(head,x)
    p ← head
    while p ≠ null
        if p.val = x
            return p
        p ← p.next
    return null
```

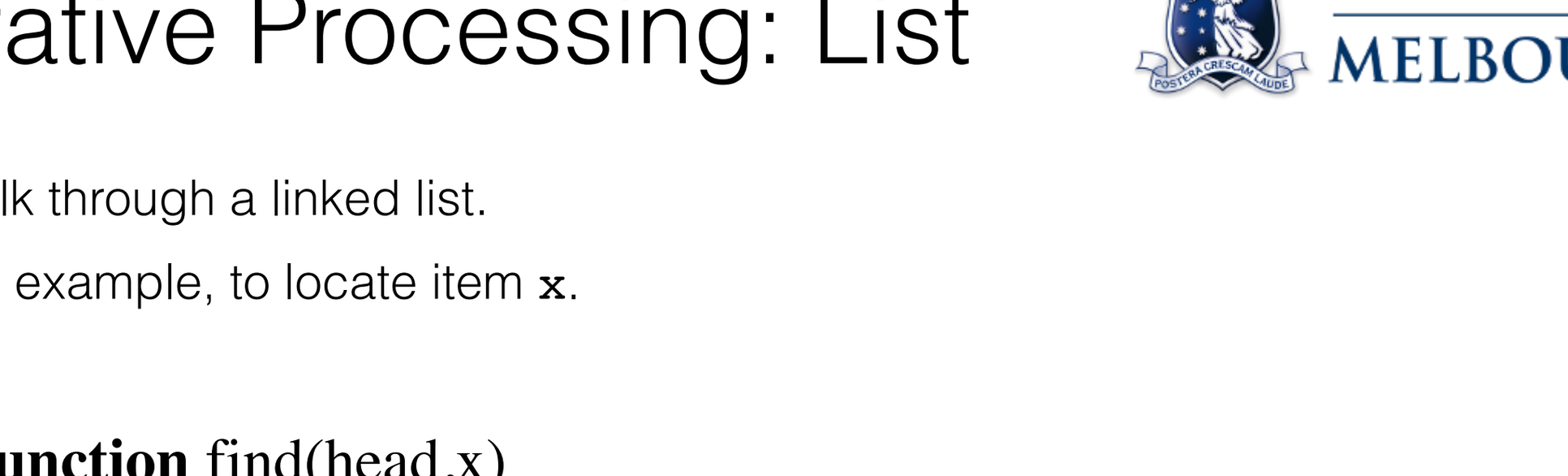


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item **`x`**.

```
function find(head,x)
    p ← head
    while p ≠ null
        if p.val = x
            return p
        p ← p.next
    return null
```

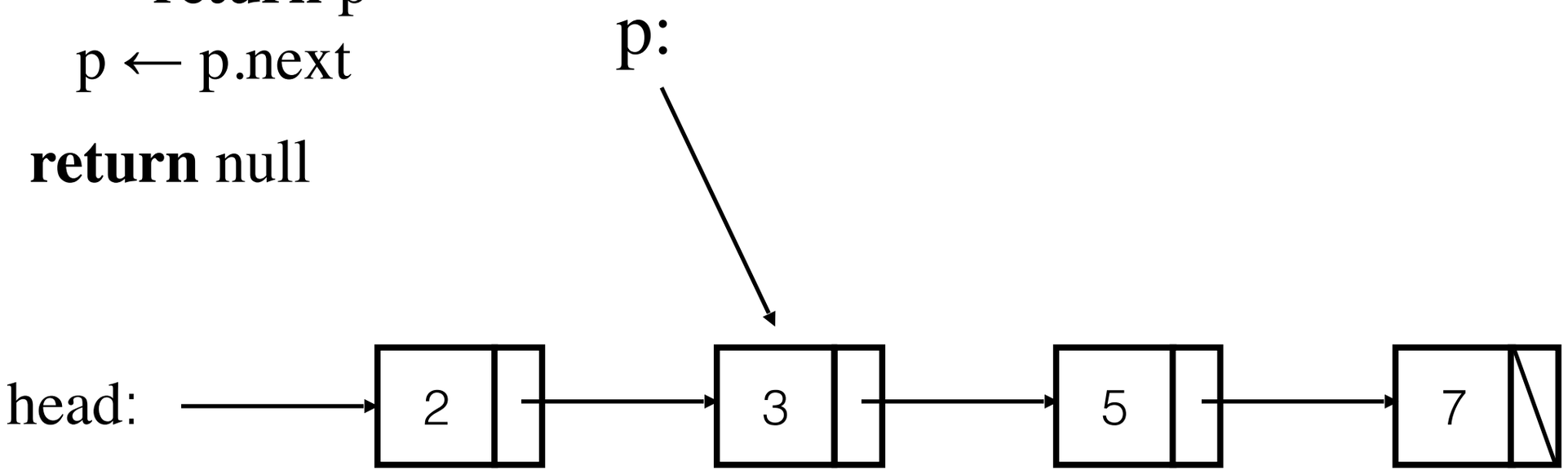


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Iterative Processing: List

- Walk through a linked list.
- For example, to locate item `x`.

```
function find(head,x)
    p ← head
    while p ≠ null
      if p.val = x
        return p
    p ← p.next
  return null
```



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(A,x,0,n-1)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```



Initial call: find(A,x,0,n-1)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Y:

6	9	2	3	7	5	8
0	1	2	3	4	5	6

Initial call: find(A,x,0,n-1)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(A,x,0,n-1)

Y:



Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

A: Y

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Y:

6	9	2	3	7	5	8
0	1	2	3	4	5	6

Initial call: find(A,x,0,n-1)

Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

A: Y x: 7

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Y:

6	9	2	3	7	5	8
0	1	2	3	4	5	6

Initial call: find(A,x,0,n-1)

Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

A: Y x: 7 lo: 0

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Y:

6	9	2	3	7	5	8
0	1	2	3	4	5	6

Initial call: find(A,x,0,n-1)

Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

A: Y x: 7 lo: 0 hi: 6

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Y:



Initial call: find(A,x,0,n-1)

Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

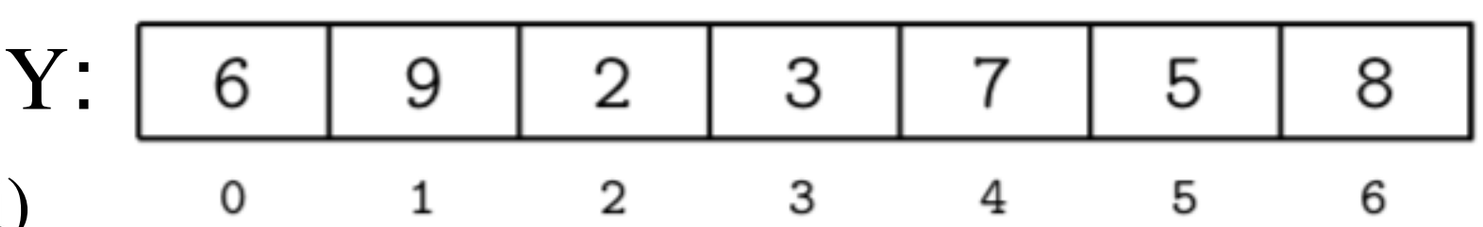
Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

A: Y x: 7 lo: 0 hi: 6

A[lo]



Initial call: find(A,x,0,n-1)

Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

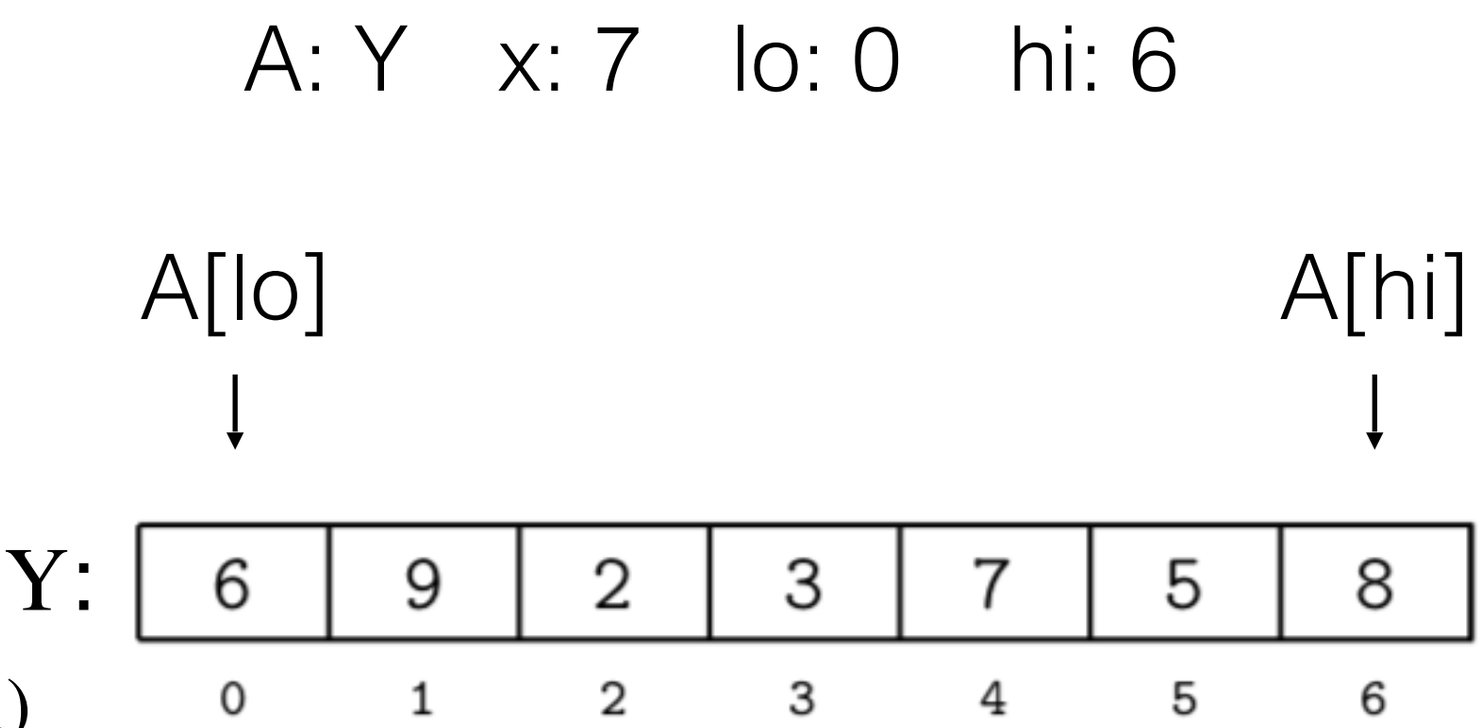
Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(A,x,0,n-1)

A: Y x: 7 lo: 0 hi: 6



Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array

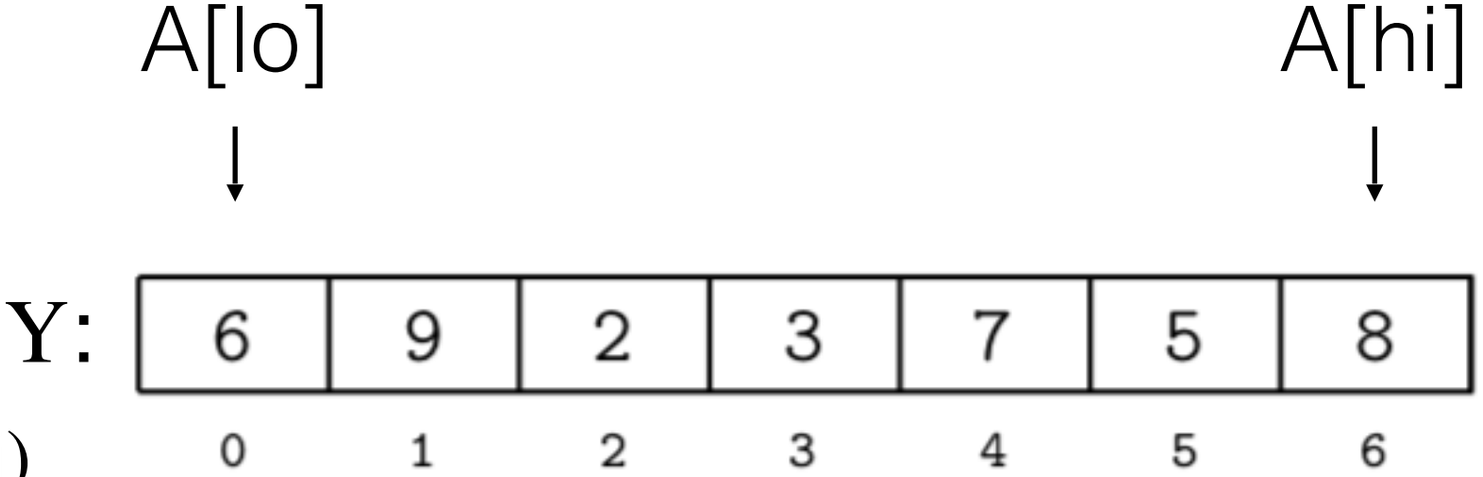
Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(A,x,0,n-1)

A: Y x: 7 lo: 1 hi: 6



Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



THE UNIVERSITY OF MELBOURNE

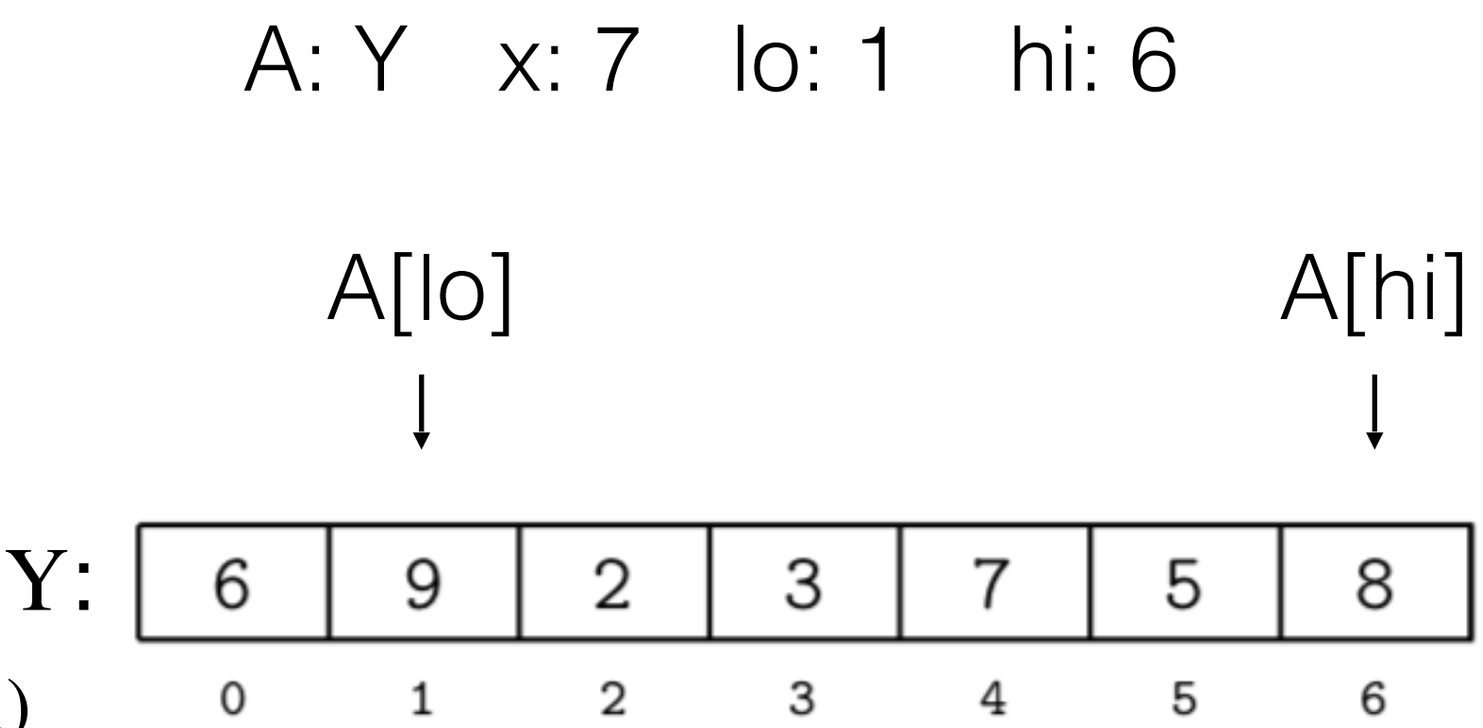
Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(A,x,0,n-1)

A: Y x: 7 lo: 1 hi: 6



Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



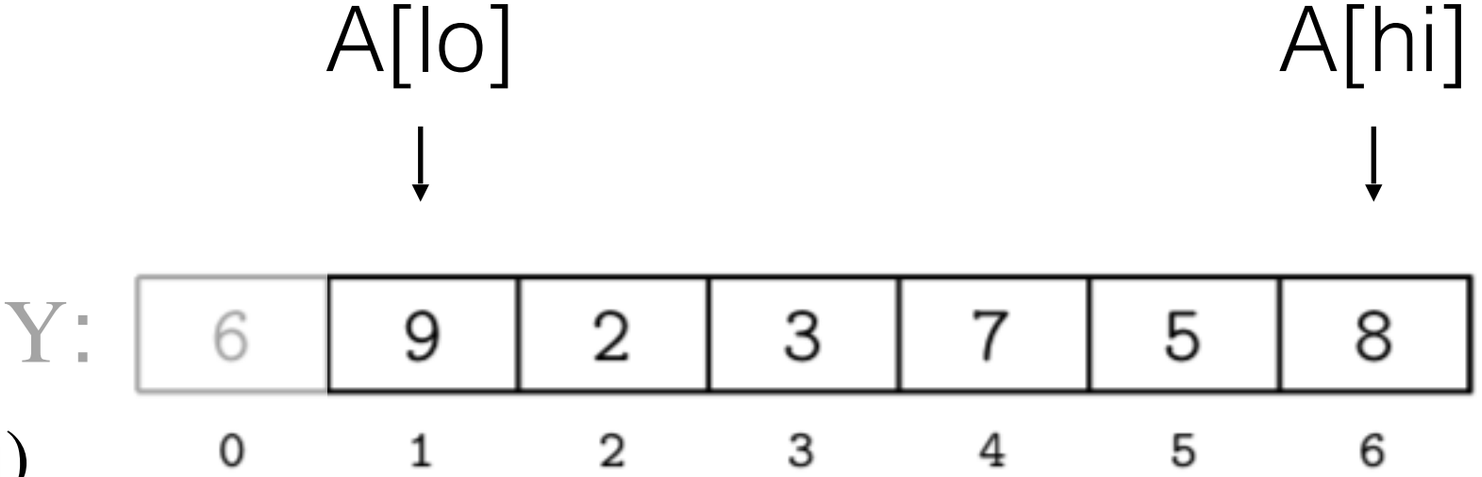
THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

A: Y x: 7 lo: 1 hi: 6



Initial call: find(A,x,0,n-1)

Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



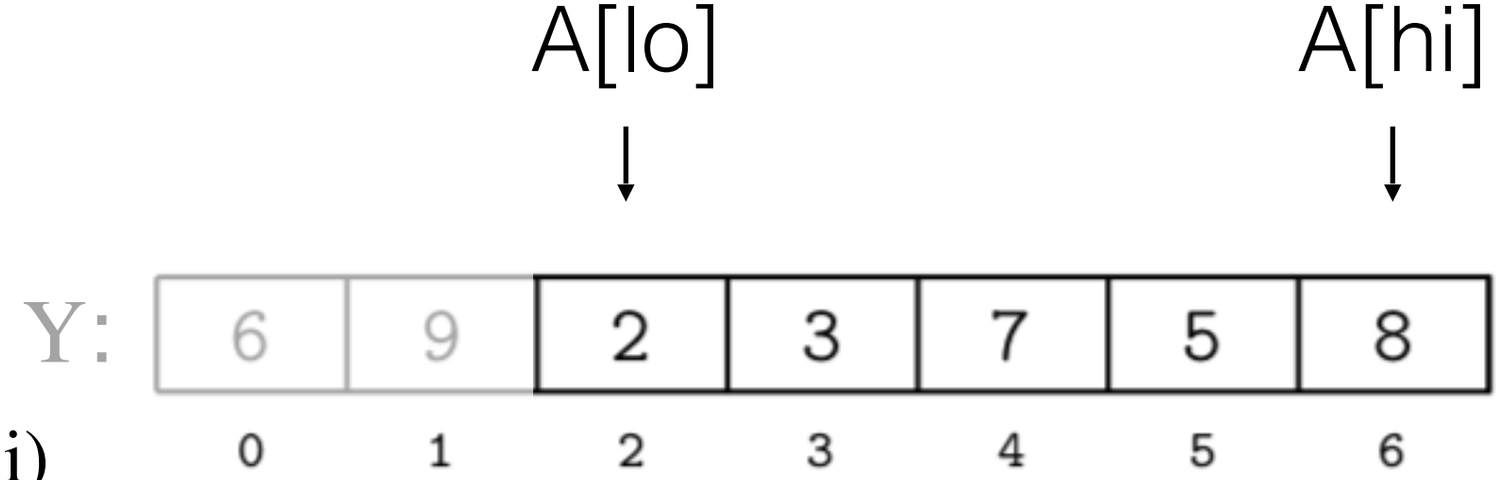
THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

A: Y x: 7 lo: 2 hi: 6

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```



Initial call: find(A,x,0,n-1)

Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



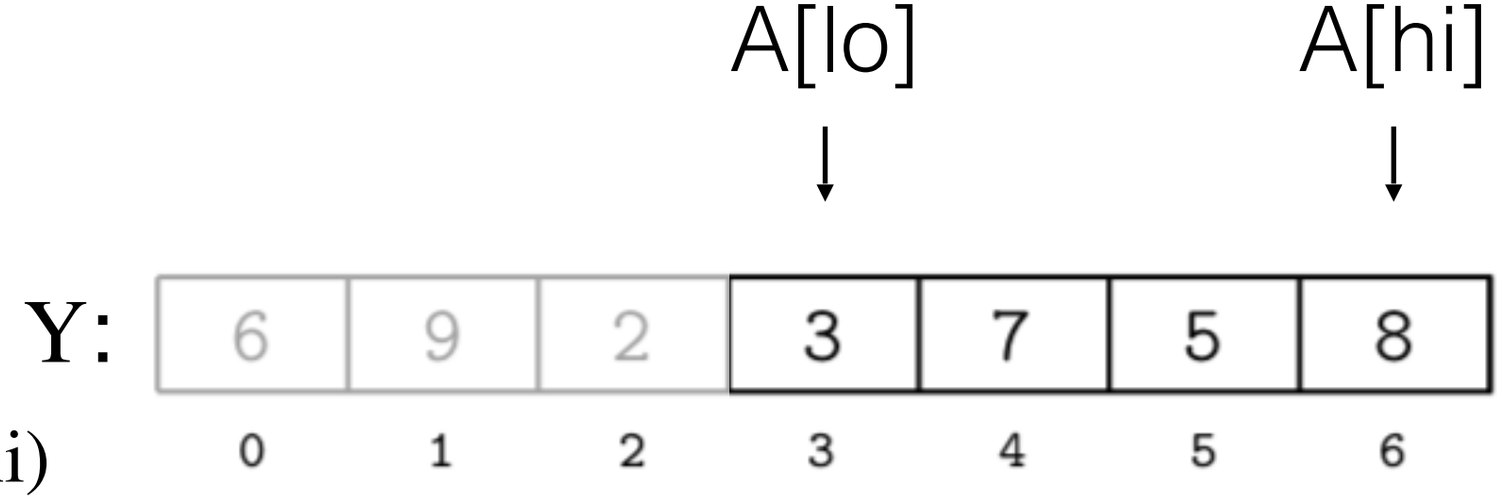
THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

A: Y x: 7 lo: 3 hi: 6

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```



Initial call: find(A,x,0,n-1)

Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array



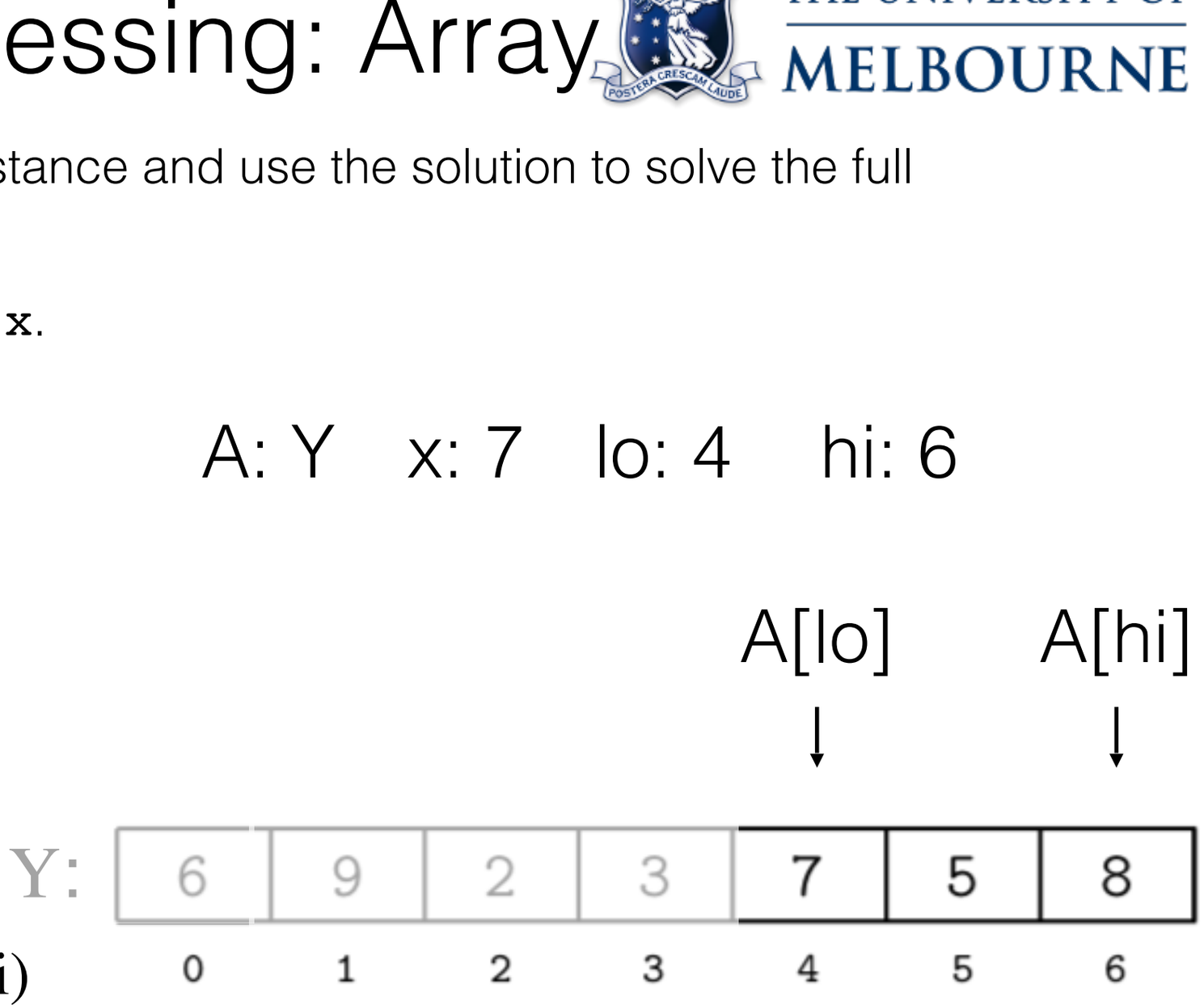
THE UNIVERSITY OF MELBOURNE

Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

A: Y x: 7 lo: 4 hi: 6

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```



Initial call: find(A,x,0,n-1)

Let's trace the execution of find(Y,7,0,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: Array

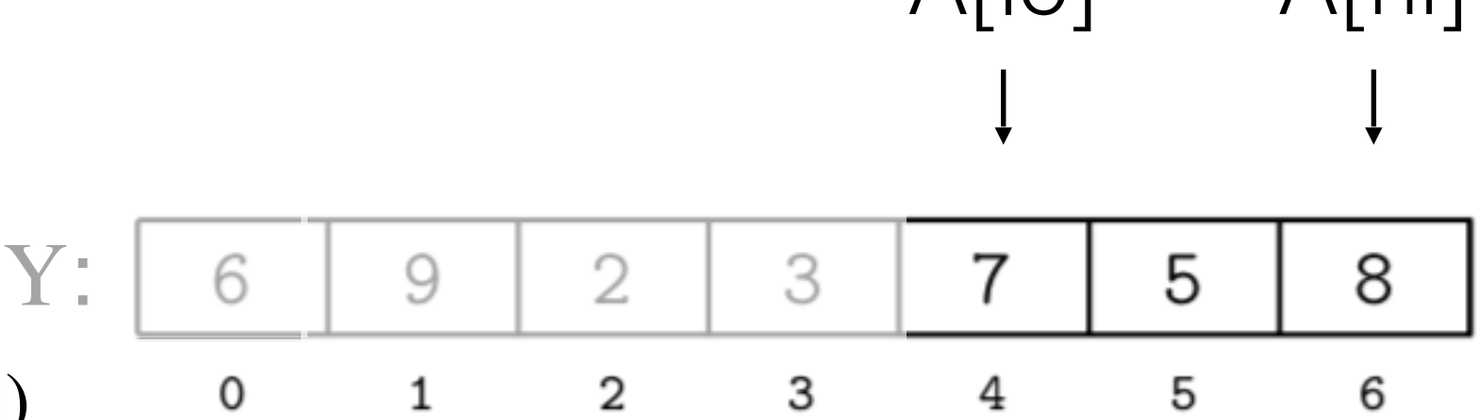
Solve the problem for a sub-instance and use the solution to solve the full instance

- For example, to locate item **x**.

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(A,x,0,n-1)

A: Y x: 7 lo: 4 hi: 6



Let's trace the execution of find(Y,7,0,6)

(returns 4)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List



THE UNIVERSITY OF MELBOURNE

- Solve the problem for a sub-instance and use the solution to solve the full instance

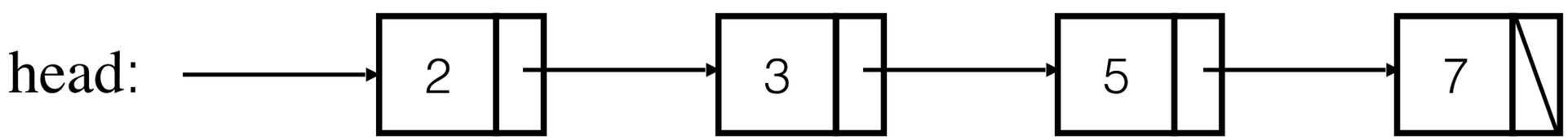
```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

- Solve the problem for a sub-instance and use the solution to solve the full instance

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```



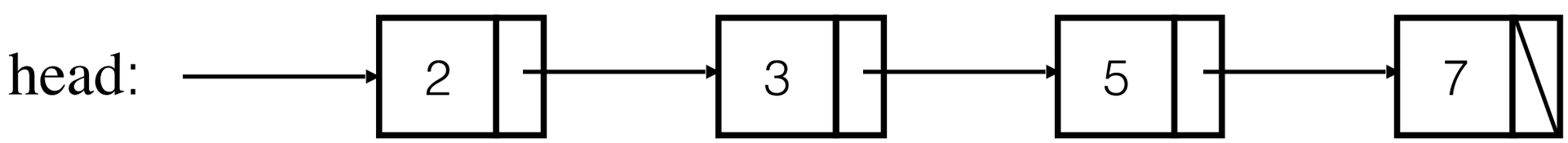
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

- Solve the problem for a sub-instance and use the solution to solve the full instance

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```

Initial call: find(head,x)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

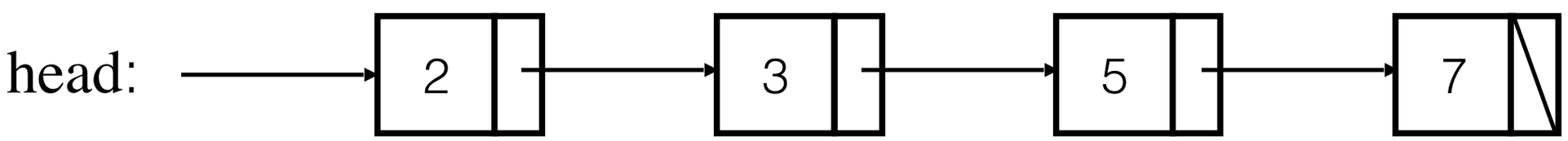
- Solve the problem for a sub-instance and use the solution to solve the full instance

(note similarity to array version)

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(head,x)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

- Solve the problem for a sub-instance and use the solution to solve the full instance

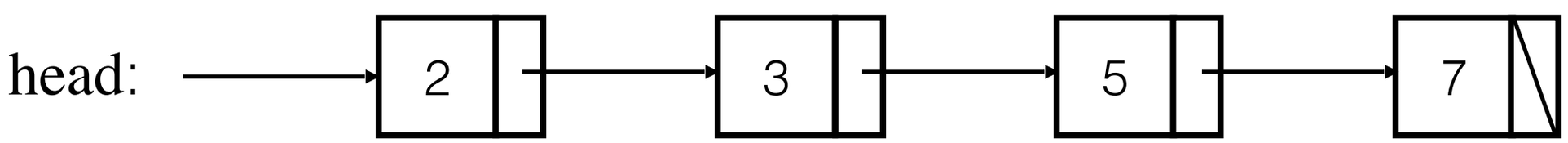
(note similarity to array version)

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

p:

Initial call: find(head,x)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

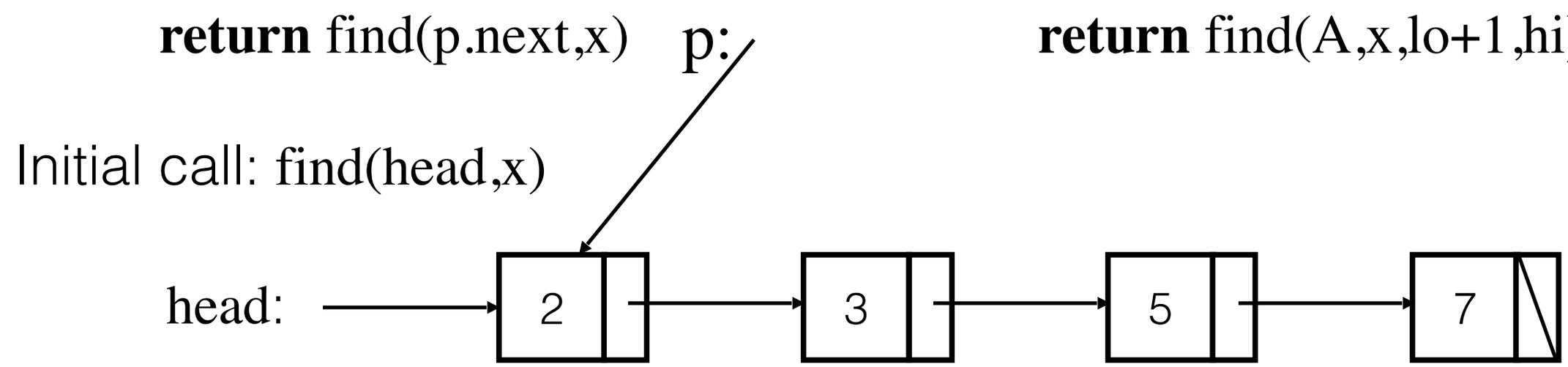
- Solve the problem for a sub-instance and use the solution to solve the full instance

(note similarity to array version)

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(head,x)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

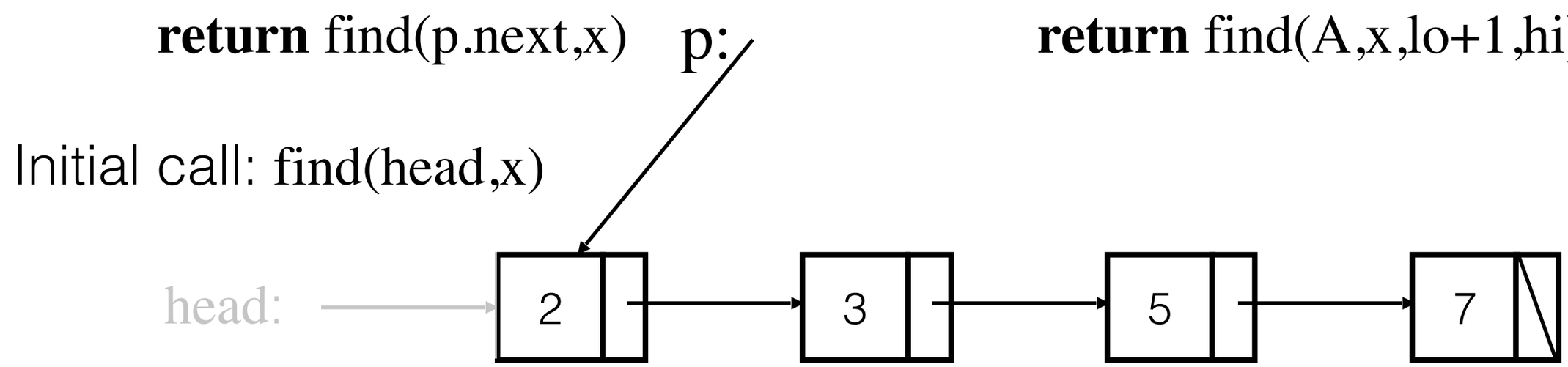
- Solve the problem for a sub-instance and use the solution to solve the full instance

(note similarity to array version)

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(head,x)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

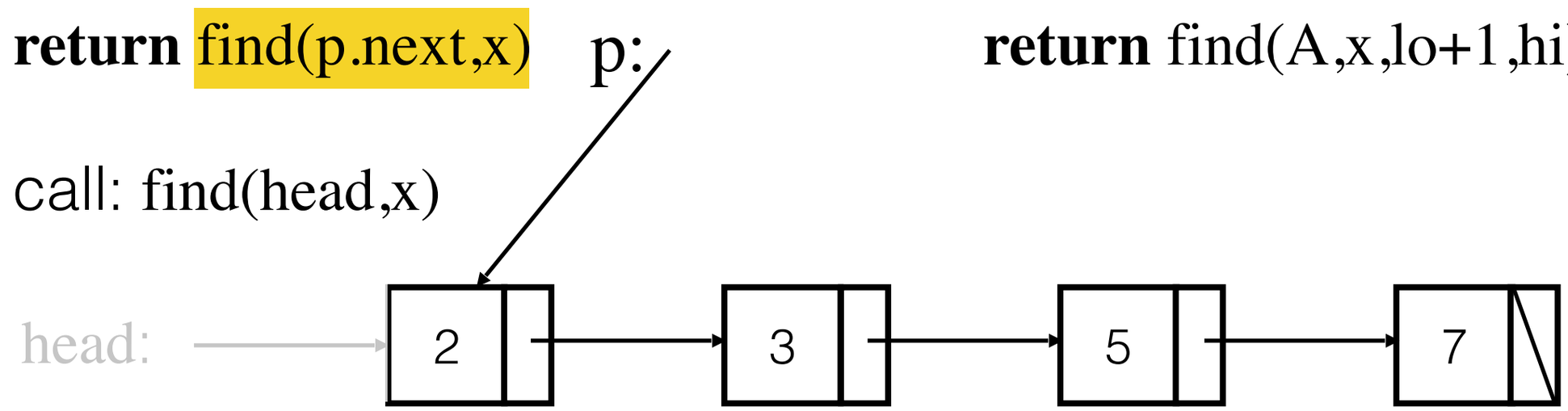
- Solve the problem for a sub-instance and use the solution to solve the full instance

(note similarity to array version)

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(head,x)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

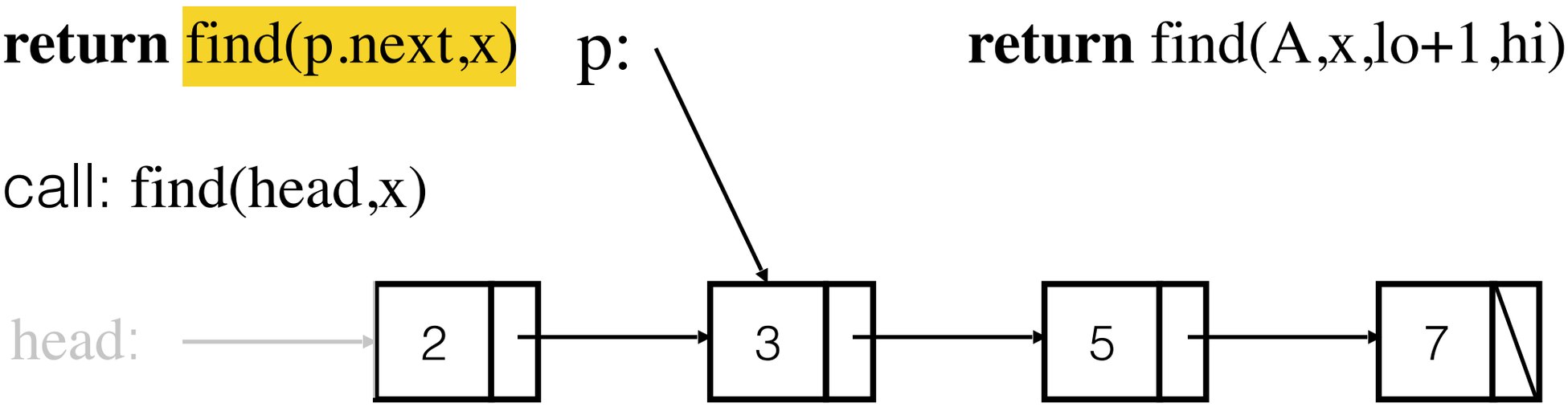
- Solve the problem for a sub-instance and use the solution to solve the full instance

(note similarity to array version)

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(head,x)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

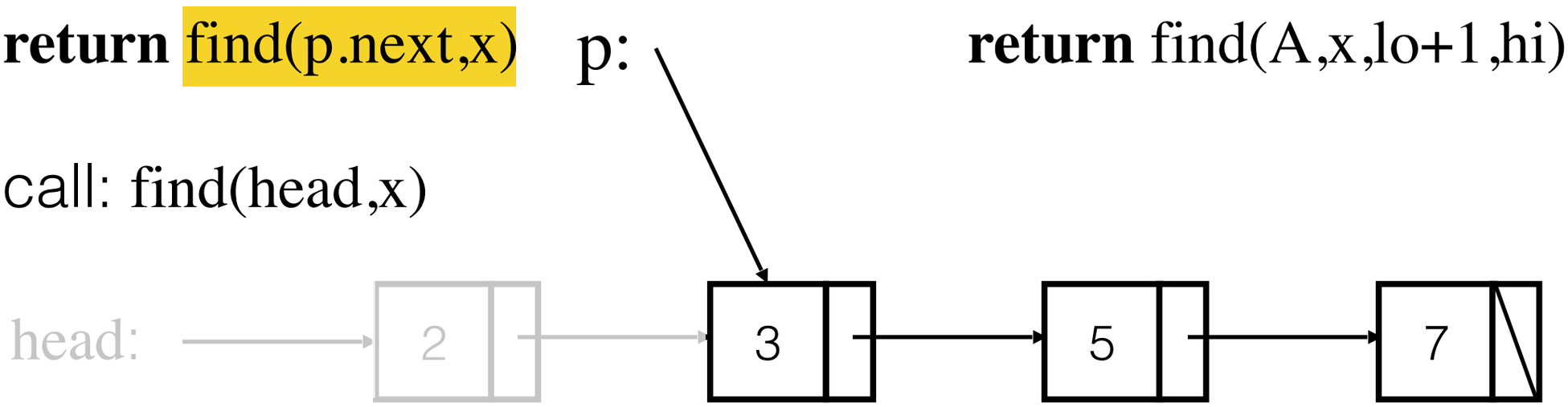
- Solve the problem for a sub-instance and use the solution to solve the full instance

(note similarity to array version)

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(head,x)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Recursive Processing: List

- Solve the problem for a sub-instance and use the solution to solve the full instance

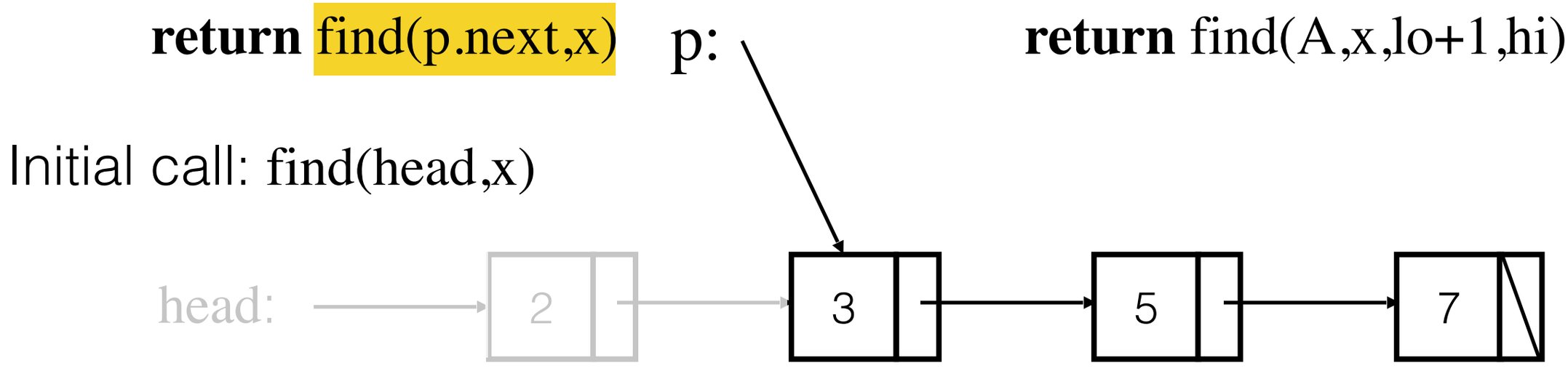
(note similarity to array version)

```
function find(p,x)
    if p = null
        return p
    else if p.val = x
        return p
    else
        return find(p.next,x)
```

We will discuss recursion properly in Week 3

```
function find(A,x,lo,hi)
    if lo > hi
        return -1
    else if A[lo] = x
        return lo
    else
        return find(A,x,lo+1,hi)
```

Initial call: find(head,x)



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Abstract DataTypes



- A collection of data items, and a family of operations that operate on that data

- Think of an ADT as a set of contracts, an **interface**

- We must still **implement** these promises, but it is an advantage to separate the implementation of the ADT from the "concept" (i.e. the interface it provides)

- Good programming practice is to support this separation

  - Nothing outside of the definition of the ADT should refer to anything inside, except through function calls and basic operations

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: The Stack



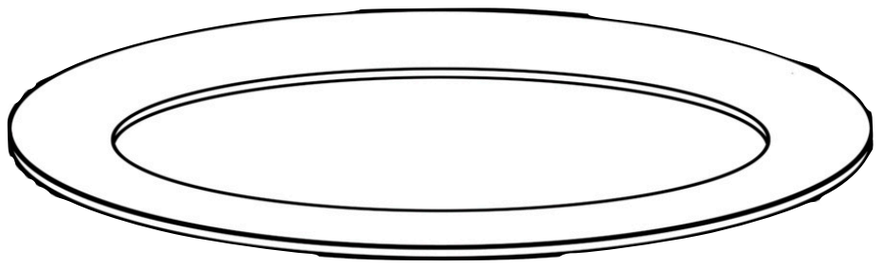
THE UNIVERSITY OF MELBOURNE

- Last-In-First-Out (LIFO)
- Operations:
  - CreateStack
  - Push
  - Pop
  - Top
  - EmptyStack?
  - …
- Usually implemented as an ADT

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: The Stack

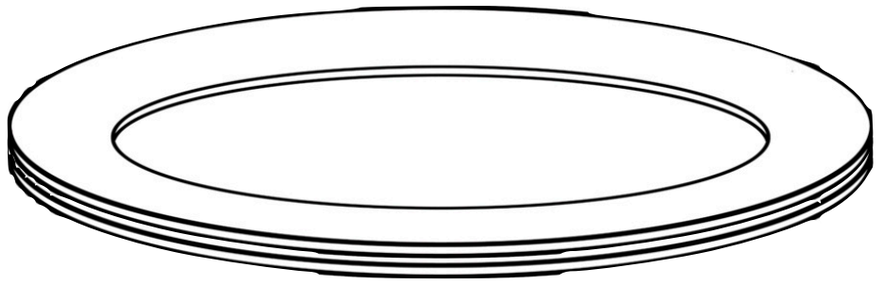
- Last-In-First-Out (LIFO)
- Operations:
  - CreateStack
  - Push
  - Pop
  - Top
  - EmptyStack?
  - …
- Usually implemented as an ADT



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: The Stack

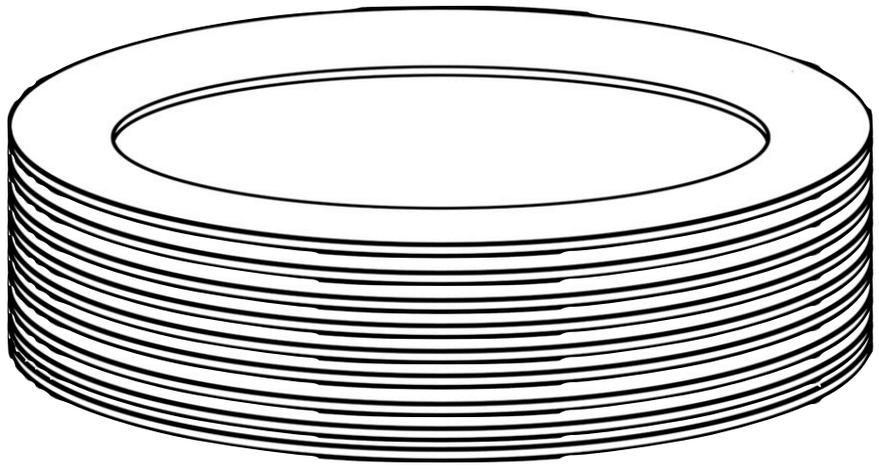
- Last-In-First-Out (LIFO)
- Operations:
  - CreateStack
  - Push
  - Pop
  - Top
  - EmptyStack?
  - …
- Usually implemented as an ADT



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: The Stack

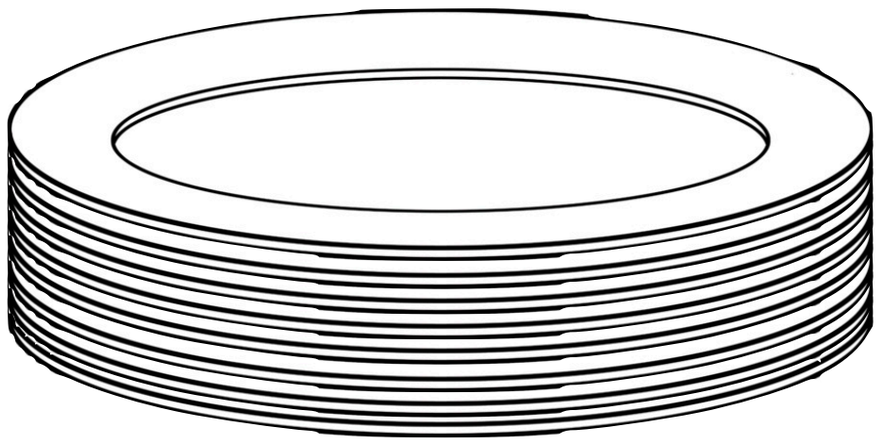
- Last-In-First-Out (LIFO)
- Operations:
  - CreateStack
  - Push
  - Pop
  - Top
  - EmptyStack?
  - …
- Usually implemented as an ADT



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: The Stack

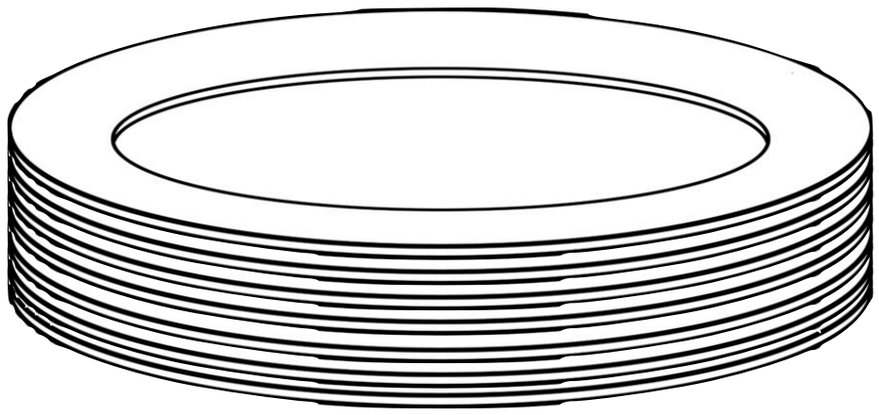
- Last-In-First-Out (LIFO)
- Operations:
  - CreateStack
  - Push
  - Pop
  - Top
  - EmptyStack?
  - …
- Usually implemented as an ADT



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: The Stack

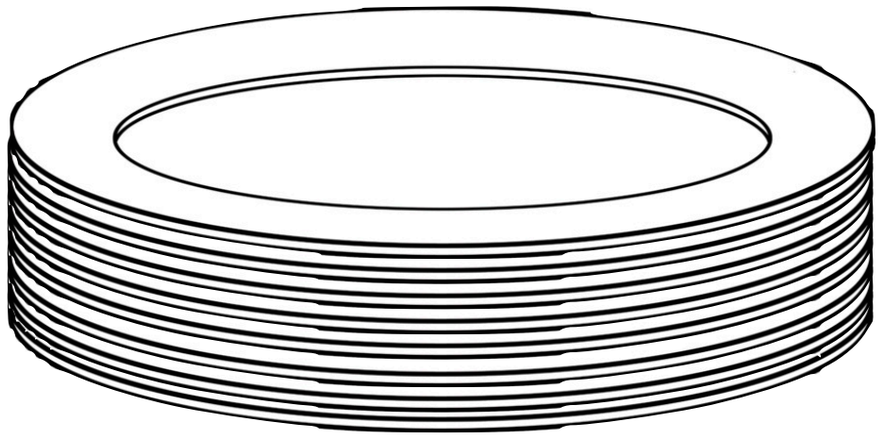
- Last-In-First-Out (LIFO)
- Operations:
  - CreateStack
  - Push
  - Pop
  - Top
  - EmptyStack?
  - …
- Usually implemented as an ADT



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: The Stack

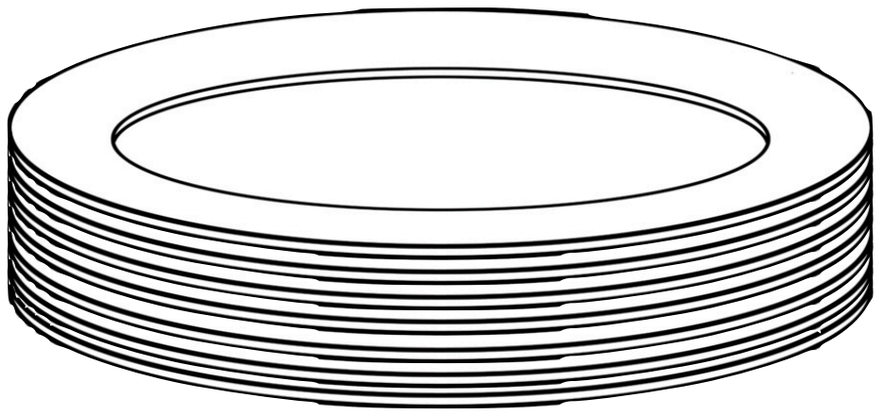
- Last-In-First-Out (LIFO)
- Operations:
  - CreateStack
  - Push
  - Pop
  - Top
  - EmptyStack?
  - …
- Usually implemented as an ADT



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: The Stack

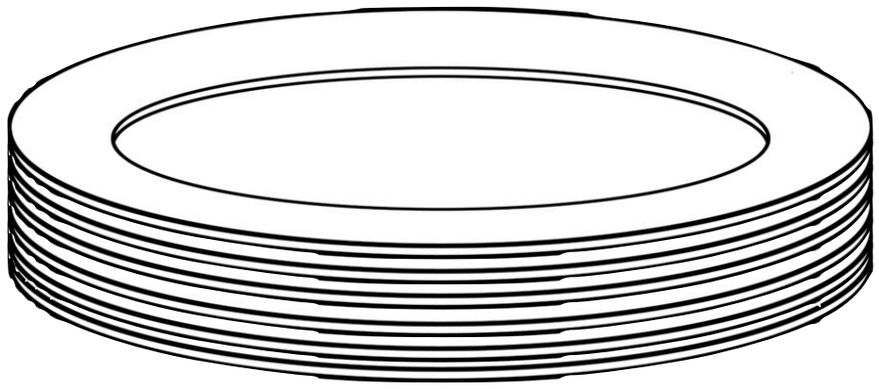
- Last-In-First-Out (LIFO)
- Operations:
  - CreateStack
  - Push
  - Pop
  - Top
  - EmptyStack?
  - …
- Usually implemented as an ADT



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: The Stack

- Last-In-First-Out (LIFO)
- Operations:
  - CreateStack
  - Push
  - Pop
  - Top
  - EmptyStack?
  - …
- Usually implemented as an ADT



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Array

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Array



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Array



top: 4

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Array



Push(7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Array



top: 4

Push(7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Array

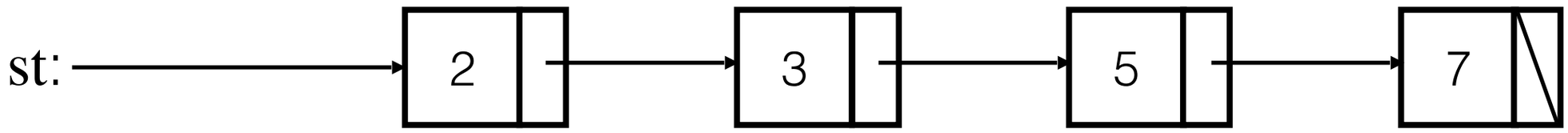


top: 5

Push(7)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List



function push(st,x)

elt ← **new** node

elt.val ← x

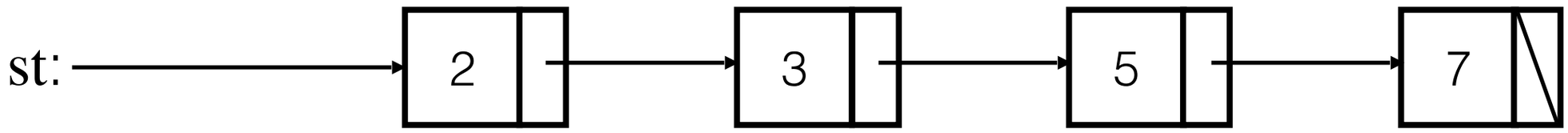
elt.next ← st

st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List



push(st,6)

function push(st,x)

elt ← **new** node

elt.val ← x

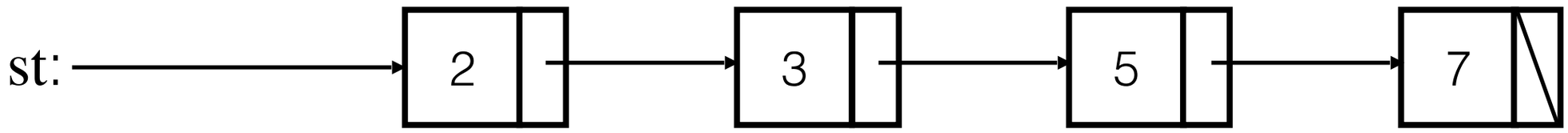
elt.next ← st

st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List



push(st,6)

function push(st,x)

elt ← **new** node

elt.val ← x

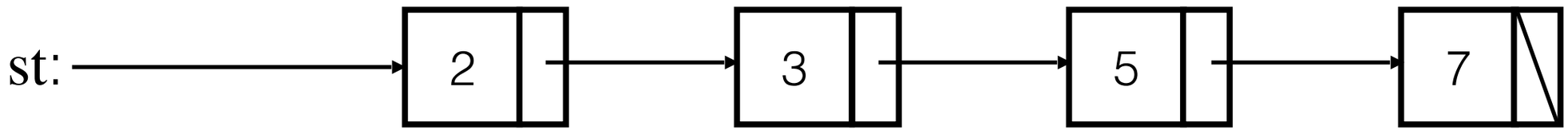
elt.next ← st

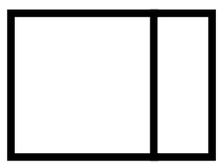
st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List





push(st,6)

function push(st,x)

elt ← **new** node

elt.val ← x

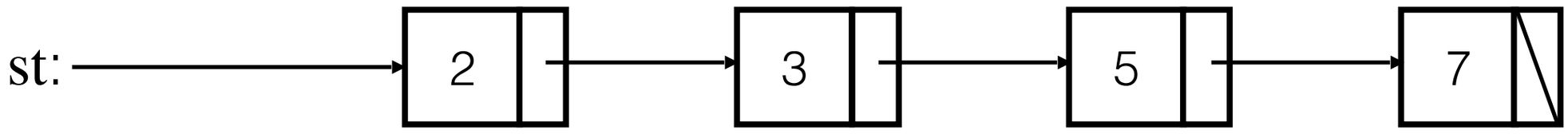
elt.next ← st

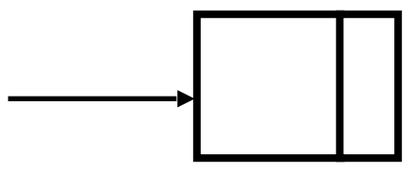
st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List





push(st,6)

function push(st,x)

elt ← **new** node

elt.val ← x

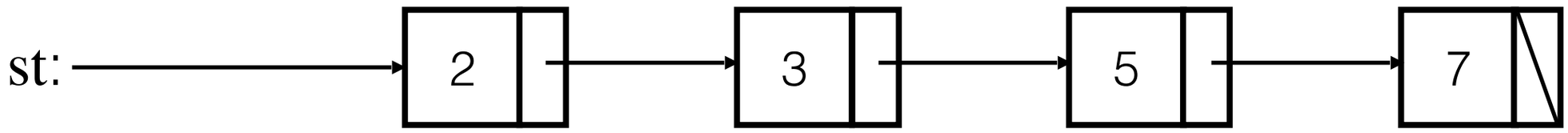
elt.next ← st

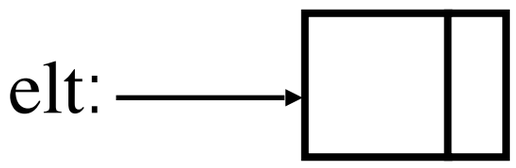
st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List





push(st,6)

function push(st,x)

elt ← **new** node

elt.val ← x

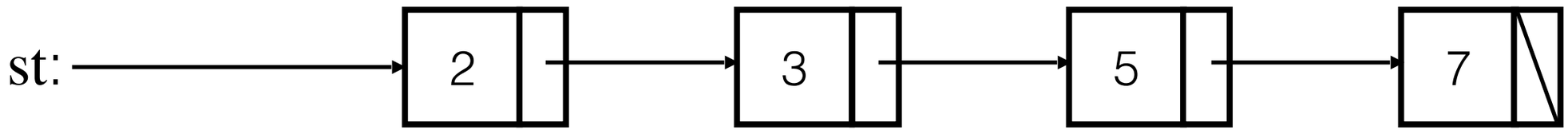
elt.next ← st

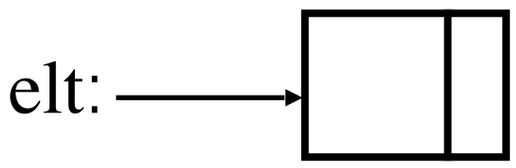
st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List





function push(st,x)

elt ← **new** node

elt.val ← x

elt.next ← st

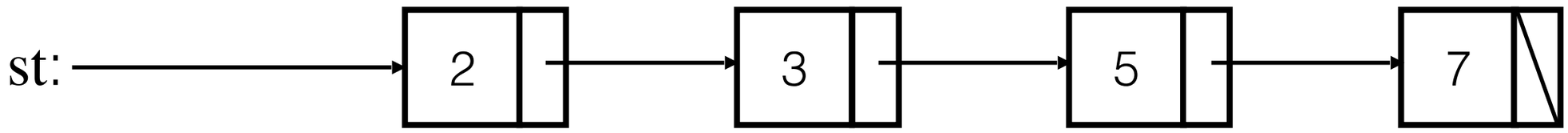
st ← elt

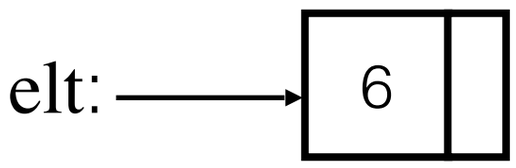
return st

push(st,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List





push(st,6)

function push(st,x)

elt ← **new** node

elt.val ← x

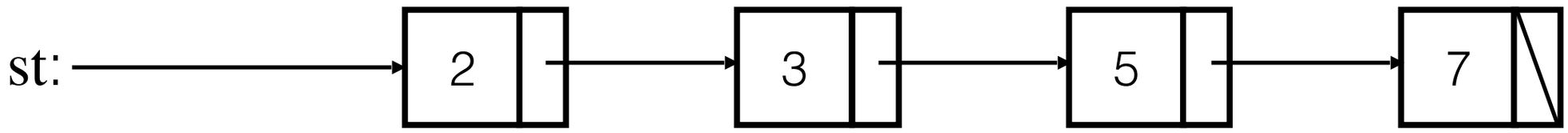
elt.next ← st

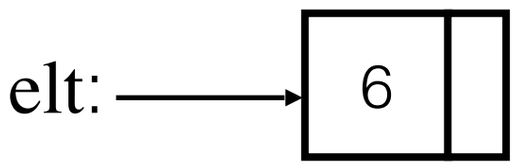
st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List





push(st,6)

function push(st,x)

elt ← **new** node

elt.val ← x

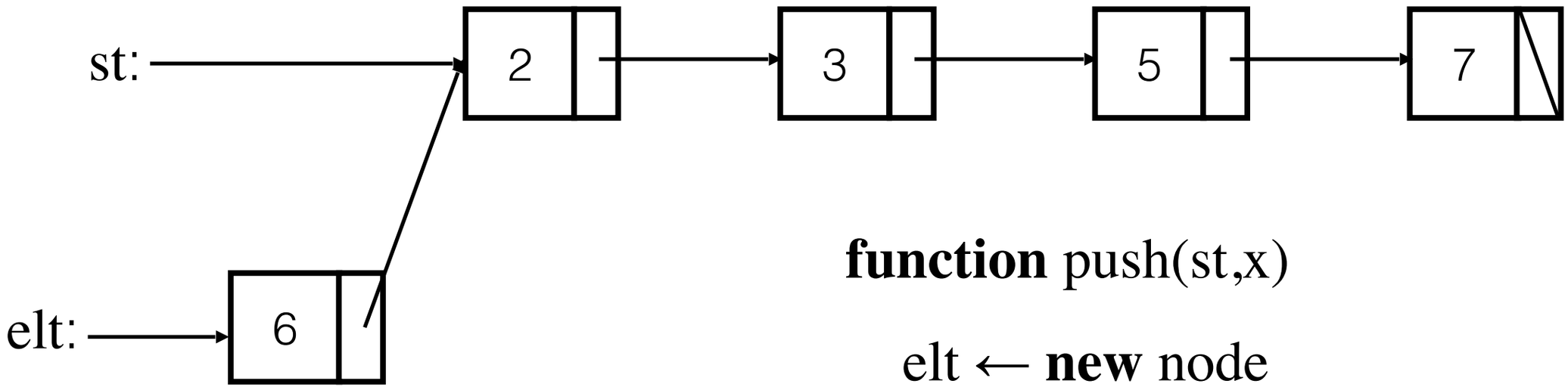
elt.next ← st

st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List



function push(st,x)

elt ← **new** node

elt.val ← x

elt.next ← st

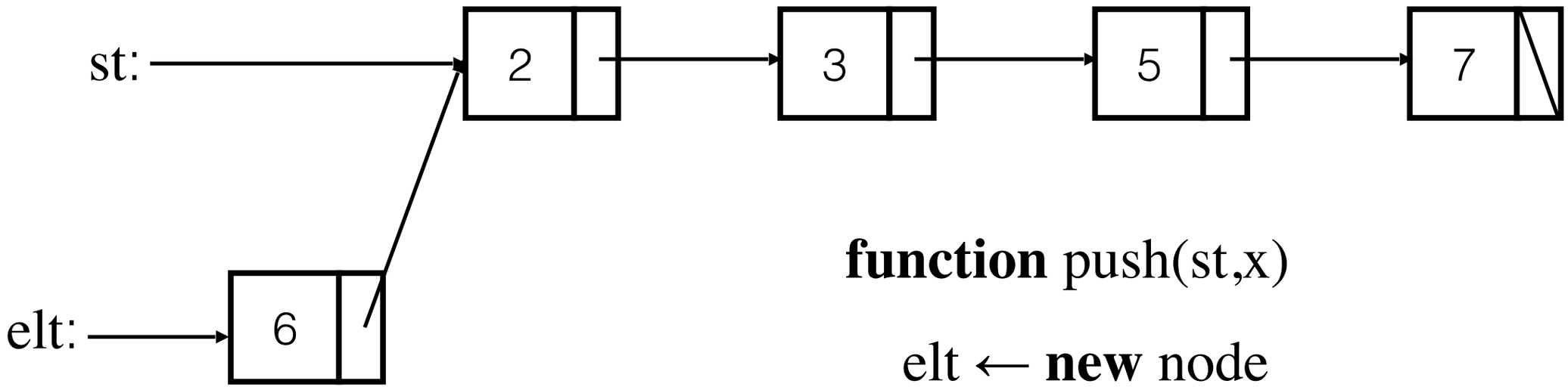
st ← elt

return st

push(st,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List



push(st,6)

function push(st,x)

elt ← **new** node

elt.val ← x

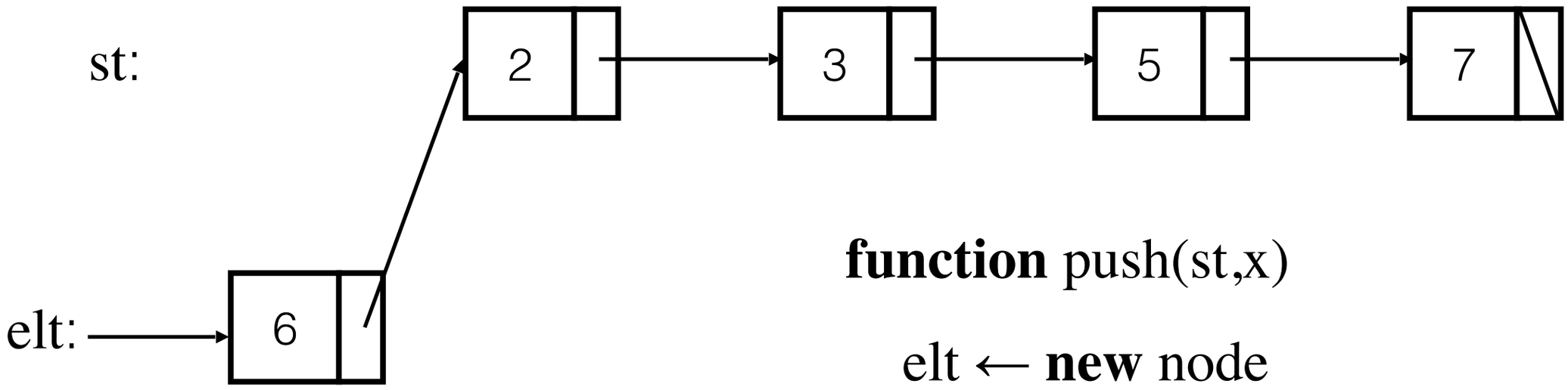
elt.next ← st

st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List



function push(st,x)

elt ← **new** node

elt.val ← x

elt.next ← st

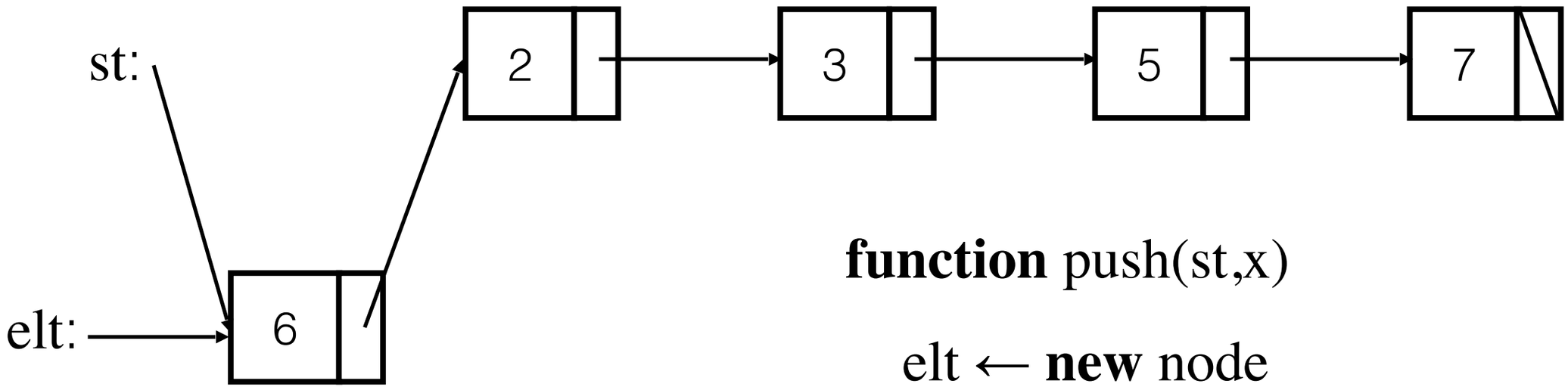
st ← elt

return st

push(st,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List



push(st,6)

function push(st,x)

elt ← **new** node

elt.val ← x

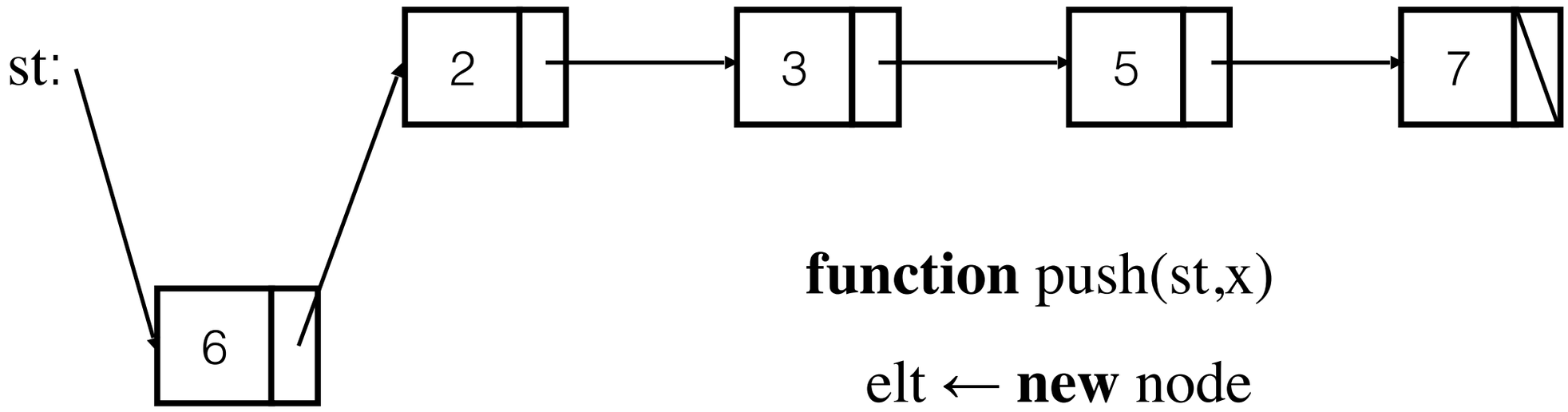
elt.next ← st

st ← elt

return st

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List

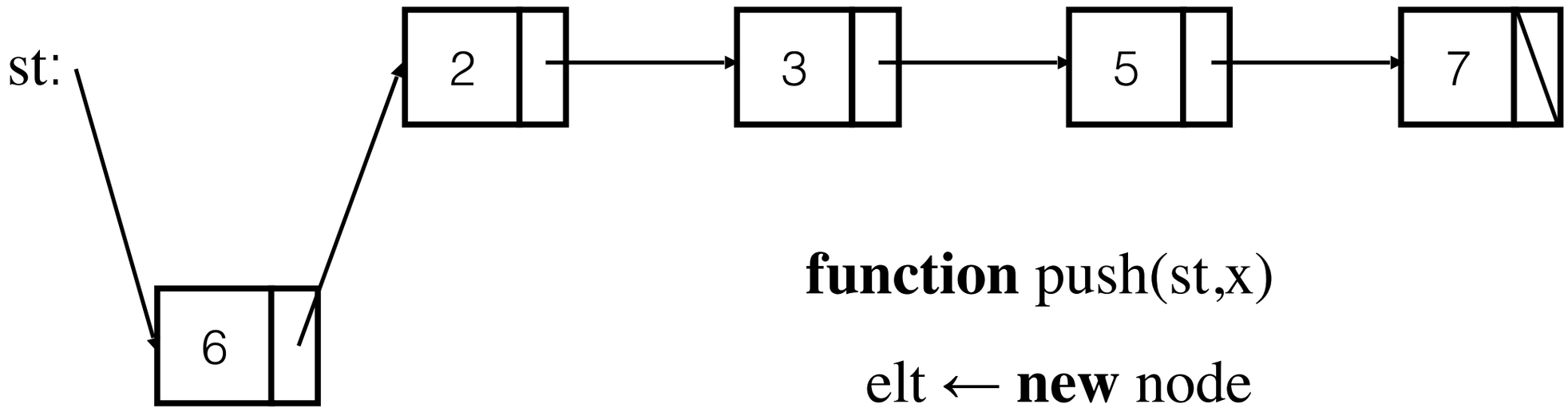



```
function push(st,x)
    elt ← new node
    elt.val ← x
    elt.next ← st
    st ← elt
    return st
```

push(st,6)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Stack Implementation: Linked List



function push(st,x)

elt ← **new** node

elt.val ← x

elt.next ← st

st ← elt

return st

push(st,6)

See
https://www.cs.usfca.edu/~galles/visualization/Algorithms.html
for more visualisations

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Pseudo Code

- On the previous slide, we assumed that a “node” has two attributes: a “val” which is its value, and a “next” which points to the rest of the list.
- There is no standard for pseudo-code. Use the examples in Levitin as a guide. Cormen et al. pages 20–22 (in Reading Resources) has a list of standard conventions used with pseudo-code which are good to follow, except we use ← as the assignment operator.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues



THE UNIVERSITY OF MELBOURNE

- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues



- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues

THE UNIVERSITY OF MELBOURNE

- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues

- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues

- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues

- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues

THE UNIVERSITY OF MELBOURNE

- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues

- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues

- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Fundamental Data Structure: Queues

- First-In-First-Out (FIFO)
- Operations:
  - CreateQueue
  - Enqueue
  - Dequeue
  - Head
  - EmptyQueue?
  - …



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Other Data Structures

- We will meet many other (abstract) data structures, e.g.
  - The priority queue
  - Various types of “tree”
  - Various types of “graph”

- If you check out algorithm animation tools or advanced algorithm books, you will meet exotic data structures such as splay trees and skip lists.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Summary and Next Week

- Today: Review of Basic concepts (arrays, lists, Abstract Data Types, stack, queue)
- Algorithm analysis—how to reason about an algorithm's resource consumption.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Announcements

Student representative call:

If you are interested in being a student representative, please sendand email to comp90038-s2-2026@unimelb.edu.au by Wednesday August 5, briefly saying what degree you are doing why you want to be a student rep (one sentence is sufficient).

Don’t forget to do Assumed Knowledge Quizzes

See consultation hours for Assumed Knowledge Quizzes on Canvas

Tutorials start Week 2

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License